

Cahier d'Exercices Rust — Maitrise Pratique

Version: 1.0

Niveau: Debutant a Avance

Prerequis: Variables, Fonctions, Types de donnees, Controle de flux, Ownership, Borrowing, References, Mutable References, Slices, Vec, String

Table des Matieres

- Introduction
- Partie 1 : 20 Exercices Progressifs
 - Exercices 1 a 5 : Debutant
 - Exercices 6 a 10 : Intermediaire
 - Exercices 11 a 15 : Ownership et Borrowing
 - Exercices 16 a 20 : Ownership, Borrowing et Slices avances
- Partie 2 : 5 Projets Complets
 - Projet 1 : Gestionnaire de notes
 - Projet 2 : Gestionnaire de taches Todo CLI
 - Projet 3 : Gestionnaire de bibliotheque
 - Projet 4 : Mini systeme bancaire
 - Projet 5 : Gestionnaire d'etudiants

Introduction

Bienvenue dans ce Cahier d'Exercices

Ce cahier d'exercices a ete concu pour transformer vos connaissances theoriques de Rust en competences pratiques solides. Il ne contient **aucun cours theorique** — seulement des exercices concrets, des projets realistes et des corrections detaillees.

Comment utiliser ce cahier

1. **Lisez l'enonce complet** avant d'ecrire la moindre ligne de code. Comprendre le probleme est la moitie de la solution.
2. **Tentez l'exercice seul** avant de consulter les indices. La frustration temporaire est le prix de l'apprentissage durable.
3. **Observez les erreurs du compilateur** Rust. Contrairement a d'autres langages, le compilateur Rust est un excellent professeur — ses messages d'erreur sont explicites et pedagogiques.
4. **Comprenez la correction** ligne par ligne. Ne vous contentez pas de faire fonctionner le code — comprenez pourquoi il fonctionne.
5. **Relevez le defi bonus** pour tester la profondeur de votre comprehension.

Regles d'or

Regle	Explication
Toujours essayer avant de regarder	Votre premier reflexe doit etre de chercher par vous-meme. La solution trouvee seule reste 10 fois plus longtemps.
Lire les erreurs du compilateur	Rust fournit des messages d' erreur exceptionnellement detailles. Lisez-les attentivement jusqu' au bout.
Comprendre le pourquoi	Ne memorisez pas le code. Comprenez pourquoi chaque reference, chaque emprunt, chaque slice est necessaire.
Experimentez	Modifiez le code de la correction, introduisez des erreurs volontairement, observez les messages du compilateur.
Pratiquez regulierement	Un exercice par jour vaut mieux que dix exercices en une seule seance.

Niveaux de difficulte

Niveau	Exercices	Description
Debutant	1 a 5	Variables, fonctions, types simples, boucles, conditions
Intermediaire	6 a 10	Vec, String, traitements de texte, calculs
Ownership et Borrowing	11 a 15	Transfert de propriete, emprunts immuables, slices de base
Avance	16 a 20	Emprunts mutables, slices avancees, combinaisons complexes

Partie 1 : 20 Exercices Progressifs

Exercices 1 a 5 : Debutant

Exercice 1 : Calculatrice de Moyenne Scolaire

Objectifs

- Declarer et initialiser des variables avec des types explicites
- Effectuer des operations arithmetiques de base
- Utiliser les fonctions avec parametres et valeur de retour
- Manipuler l'affichage avec `println!`

Enonce

Vous devez creer un programme qui calcule la moyenne generale d'un eleve a partir de ses notes dans cinq matieres differentes. Le programme doit egalement determiner si l'eleve est admis (moyenne ≥ 10) ou non.

Questions

1. Declarez cinq variables representant les notes dans les matieres : Mathematiques, Physique, Chimie, Biologie et Informatique. Attribuez les valeurs 14.5, 12.0, 8.5, 15.0 et 17.0 respectivement.
2. Ecrivez une fonction `calculer_moyenne` qui prend cinq parametres de type `f64` et retourne la moyenne.
3. Ecrivez une fonction `est_admis` qui prend la moyenne en parametre et retourne un boolean.
4. Affichez chaque note individuellement avec le nom de la matiere.
5. Affichez la moyenne generale avec deux decimales.
6. Affichez un message indiquant si l'eleve est admis ou doit passer en rattrapage.

Indices

- Le type `f64` est le type flottant a double precision en Rust.
- Pour formater un nombre avec deux decimales, utilisez `{:.2}` dans `println!`.
- La moyenne est la somme des notes divisee par le nombre de matieres.
- Une fonction retourne la derniere expression sans point-virgule (ou utilisez `return`).

Correction complete

```
fn main() {  
    // Declaration des cinq notes avec type explicite f64  
    let maths: f64 = 14.5;  
    let physique: f64 = 12.0;  
    let chimie: f64 = 8.5;  
    let biologie: f64 = 15.0;  
    let informatique: f64 = 17.0;  
  
    // Affichage de chaque note individuelle  
    println!("Mathematiques : {:.2}", maths);  
    println!("Physique : {:.2}", physique);  
    println!("Chimie : {:.2}", chimie);  
    println!("Biologie : {:.2}", biologie);  
    println!("Informatique : {:.2}", informatique);  
  
    // Appel de la fonction calculer_moyenne avec les cinq notes  
    let moyenne = calculer_moyenne(maths, physique, chimie, biologie, informatique);  
  
    // Affichage de la moyenne avec deux decimales  
    println!("\nMoyenne generale : {:.2}", moyenne);  
  
    // Verification de l'admission  
    if est_admis(moyenne) {  
        println!("Resultat : ADMIS");  
    } else {  
        println!("Resultat : RATRAPAGE");  
    }  
}  
  
/// Calcule la moyenne de cinq notes  
/// Parametres :
```

```

/// - n1, n2, n3, n4, n5 : des f64 representant les notes
/// Retourne :
/// - Un f64 representant la moyenne arithmetique
fn calculer_moyenne(n1: f64, n2: f64, n3: f64, n4: f64, n5: f64) -> f64 {
    let somme = n1 + n2 + n3 + n4 + n5;
    somme / 5.0
}

/// Determine si un eleve est admis
/// Parametre :
/// - moyenne : un f64 representant la moyenne
/// Retourne :
/// - true si la moyenne >= 10.0, false sinon
fn est_admis(moyenne: f64) -> bool {
    moyenne >= 10.0
}

```

Explication detaillee ligne par ligne

Ligne 1 : `fn main() {` — Point d'entree du programme. Tout executable Rust commence par la fonction `main`.

Ligne 3 : `let maths: f64 = 14.5;` — Declaration d'une variable immuable `maths` de type `f64` (nombre flottant 64 bits) initialisee a 14.5. Le mot-cle `let` declare une nouvelle variable. Le type `f64` est explicitement annoté apres les deux-points.

Lignes 4-7 : Declarations similaires pour les autres matieres. Chaque variable est immuable par default en Rust, ce qui signifie qu'on ne peut pas la reassigner apres initialisation.

Lignes 10-14 : `println!("Mathematiques : {:.2}", maths);` — Macro d'affichage. Le format `{:.2}` indique d'afficher le nombre avec exactement 2 chiffres apres la virgule.

Ligne 17 : `let moyenne = calculer_moyenne(...);` — Appel de la fonction avec les cinq notes comme arguments. La valeur retournee est stockee dans la variable `moyenne`. Le type est infere automatiquement comme `f64`.

Ligne 23-27 : Structure conditionnelle `if/else`. La fonction `est_admis` retourne un booleen qui determine quel bloc s'execute.

Ligne 33 : `fn calculer_moyenne(n1: f64, n2: f64, n3: f64, n4: f64, n5: f64) -> f64 {` — Declaration d'une fonction avec cinq parametres de type `f64` et un type de retour `f64` (indique par la fleche `->`).

Ligne 34 : `let somme = n1 + n2 + n3 + n4 + n5;` — Addition des cinq parametres. Le resultat est stocke dans la variable `somme`.

Ligne 35 : `somme / 5.0` — Expression de retour (sans point-virgule). La division d'un `f64` par un `f64` donne un `f64`. Notez le `5.0` et non `5` — en Rust, on ne peut pas melanger entiers et flottants dans une operation.

Ligne 42 : `fn est_admis(moyenne: f64) -> bool {` — Fonction prenant un `f64` et retournant un `bool` (booleen).

Ligne 43 : `moyenne >= 10.0` — Expression de comparaison qui evalue a `true` ou `false`.

Bibliothèques et fonctions utilisées

Fonction/Macro	Rôle	Paramètres	Valeur retournée	Exemple supplémentaire
<code>println!</code>	Affiche du texte suivi d'un retour à la ligne	Format string + arguments	Aucune <code>()</code>	<code>println!("{}", 42);</code>
<code>{:.2}</code>	Spécificateur de format pour 2 décimales	Nombre flottant	Chaîne formatée	<code>println!("{:.4}", 3.14159);</code> affiche 3.1416

Pièges fréquents

Erreur	Message du compilateur	Explication
Oublier le type <code>f64</code>	<code>cannot infer the type</code>	Rust ne peut pas deviner si c'est <code>f32</code> ou <code>f64</code> .
Utiliser 5 au lieu de <code>5.0</code>	<code>cannot divide "f64" by "{integer}"</code>	Rust refuse de mélanger les types numériques implicitement.
Point-virgule après l'expression de retour	<code>expected "f64", found "()"</code>	Le point-virgule transforme l'expression en instruction qui retourne <code>()</code> .
<code>let maths = 14.5;</code> <code>maths = 16.0;</code>	<code>cannot assign twice to immutable variable</code>	Les variables sont immuables par défaut en Rust.

Defi bonus

Ajoutez une fonction `mention` qui retourne une chaîne de caractères (`String`) selon la règle suivante :

- Moyenne ≥ 16 : “Tres Bien”
- Moyenne ≥ 14 : “Bien”
- Moyenne ≥ 12 : “Assez Bien”
- Moyenne ≥ 10 : “Passable”
- Sinon : “Insuffisant”

Exercice 2 : Convertisseur de Temperatures

Objectifs

- Comprendre les conversions de types numeriques
- Utiliser les fonctions avec retour de valeurs
- Pratiquer les operations arithmetiques
- Manipuler les chaines de caracteres simples

Enonce

Creez un programme de conversion de temperatures entre Celsius, Fahrenheit et Kelvin. Le programme doit permettre de convertir dans les deux sens entre chaque unite.

Questions

1. Ecrivez une fonction `celsius_to_fahrenheit` qui convertit Celsius en Fahrenheit. Formule : $F = C * 9/5 + 32$
2. Ecrivez une fonction `fahrenheit_to_celsius` qui convertit Fahrenheit en Celsius. Formule : $C = (F - 32) * 5/9$
3. Ecrivez une fonction `celsius_to_kelvin` qui convertit Celsius en Kelvin. Formule : $K = C + 273.15$
4. Ecrivez une fonction `kelvin_to_celsius` qui convertit Kelvin en Celsius. Formule : $C = K - 273.15$
5. Ecrivez une fonction `fahrenheit_to_kelvin` qui utilise les deux fonctions precedentes.
6. Ecrivez une fonction `kelvin_to_fahrenheit` qui utilise les deux fonctions precedentes.
7. Dans le main, testez toutes ces fonctions avec la valeur 100.0 et affichez les resultats.

Indices

- Les temperatures utilisent le type `f64` pour la precision.
- Vous pouvez composer les fonctions : `fahrenheit_to_kelvin` peut appeler `fahrenheit_to_celsius` puis `celsius_to_kelvin`.
- Faites attention a l'ordre des operations dans les formules.

Correction complete

```
fn main() {  
    let temp_celsius: f64 = 100.0;  
  
    // Conversion Celsius vers Fahrenheit  
    let temp_f = celsius_to_fahrenheit(temp_celsius);  
    println!("{:.2} C = {:.2} F", temp_celsius, temp_f);  
  
    // Conversion Fahrenheit vers Celsius  
    let temp_c = fahrenheit_to_celsius(temp_f);  
    println!("{:.2} F = {:.2} C", temp_f, temp_c);  
  
    // Conversion Celsius vers Kelvin  
    let temp_k = celsius_to_kelvin(temp_celsius);  
    println!("{:.2} C = {:.2} K", temp_celsius, temp_k);  
  
    // Conversion Kelvin vers Celsius  
    let temp_c2 = kelvin_to_celsius(temp_k);
```

```

println!("{:.2} K = {:.2} C", temp_k, temp_c2);

// Conversion Fahrenheit vers Kelvin (composition)
let temp_k2 = fahrenheit_to_kelvin(temp_f);
println!("{:.2} F = {:.2} K", temp_f, temp_k2);

// Conversion Kelvin vers Fahrenheit (composition)
let temp_f2 = kelvin_to_fahrenheit(temp_k);
println!("{:.2} K = {:.2} F", temp_k, temp_f2);

// Verification : reconvertir doit donner la valeur initiale
println("\nVerification : {:.2} C -> {:.2} F -> {:.2} C",
        temp_celsius, temp_f, fahrenheit_to_celsius(temp_f));
}

/// Convertit Celsius en Fahrenheit
/// Formule : F = C * 9/5 + 32
fn celsius_to_fahrenheit(c: f64) -> f64 {
    c * 9.0 / 5.0 + 32.0
}

/// Convertit Fahrenheit en Celsius
/// Formule : C = (F - 32) * 5/9
fn fahrenheit_to_celsius(f: f64) -> f64 {
    (f - 32.0) * 5.0 / 9.0
}

/// Convertit Celsius en Kelvin
/// Formule : K = C + 273.15
fn celsius_to_kelvin(c: f64) -> f64 {
    c + 273.15
}

/// Convertit Kelvin en Celsius
/// Formule : C = K - 273.15
fn kelvin_to_celsius(k: f64) -> f64 {
    k - 273.15
}

/// Convertit Fahrenheit en Kelvin en composant les fonctions
fn fahrenheit_to_kelvin(f: f64) -> f64 {
    let c = fahrenheit_to_celsius(f);
    celsius_to_kelvin(c)
}

/// Convertit Kelvin en Fahrenheit en composant les fonctions
fn kelvin_to_fahrenheit(k: f64) -> f64 {
    let c = kelvin_to_celsius(k);
    celsius_to_fahrenheit(c)
}

```

Explication detaillee ligne par ligne

Ligne 2 : `let temp_celsius: f64 = 100.0;` — Variable `temp_celsius` de type `f64` initialisee a 100.0. C'est notre temperature de test.

Ligne 5 : `let temp_f = celsius_to_fahrenheit(temp_celsius);` — Appel de la fonction avec `temp_celsius` comme argument. La valeur 100.0 est **copiee** dans le parametre `c` de la fonction (les `f64` implementent le trait `Copy`).

Ligne 33 : `fn celsius_to_fahrenheit(c: f64) -> f64 {` — La fonction prend un parametre nomme `c` de type `f64`. Ce parametre est une variable locale a la fonction.

Ligne 34 : `c * 9.0 / 5.0 + 32.0` — Expression arithmetique. L'ordre des operations respecte les priorites mathematiques standard : multiplication et division d'abord, puis addition. Les constantes `9.0`, `5.0` et `32.0` sont toutes de type `f64`.

Lignes 48-51 : `fahrenheit_to_kelvin` montre la **composition de fonctions**. On appelle d'abord `fahrenheit_to_celsius` pour obtenir la temperature en Celsius, puis on passe ce resultat a `celsius_to_kelvin`. La variable intermediaire `c` stocke le resultat de la premiere conversion.

Lignes 54-57 : `kelvin_to_fahrenheit` utilise la meme technique de composition mais dans l'ordre inverse.

Lignes 22-24 : La verification affiche la chaine de conversion Celsius -> Fahrenheit -> Celsius pour s'assurer qu'on retrouve bien la valeur initiale.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>println!</code>	Affichage formate	Format + arguments	<code>()</code>	<code>println!("{}", 3.14159);</code>

Pieges frequents

Erreur	Message du compilateur	Explication
9/5 au lieu de 9.0/5.0	<code>cannot divide "f64" by "{integer}"</code>	En Rust, 9 est un entier, 9.0 est un flottant. Pas de conversion implicite.
<code>c * (9/5) + 32</code>	Resultat incorrect : 37.0	9/5 en entier vaut 1 (division entiere). Il faut 9.0/5.0.
<code>(f - 32) * 5/9</code> sans parentheses	<code>expected "f64", found "()"</code>	L' ordre des operations est change sans les parentheses.

Defi bonus

Ajoutez une fonction `formater_temperature` qui prend une valeur et une unite (`&str`) et retourne une `String` formatee comme `"42.50 C"`. Puis utilisez-la pour simplifier tous les `println!` du `main`.

Exercice 3 : Compteur de Voyelles et Consonnes

Objectifs

- Manipuler les chaines de caracteres (`String` et `&str`)
- Utiliser les boucles `for` pour iterer sur des caracteres
- Comprendre les conditions avec `if/else if/else`
- Introduire les notions de reference (`&str`)

Enonce

Ecrivez un programme qui analyse une phrase donnee et compte le nombre de voyelles, de consonnes, d'espaces et d'autres caracteres.

Questions

1. Declarez une variable `phrase` de type `&str` contenant "Le langage Rust est fascinant et puissant!".
2. Ecrivez une fonction `compter_voyelles` qui prend une reference sur une chaine et retourne le nombre de voyelles (a, e, i, o, u, y et leurs majuscules).
3. Ecrivez une fonction `compter_consonnes` qui retourne le nombre de lettres qui ne sont pas des voyelles.
4. Ecrivez une fonction `compter_espaces` qui retourne le nombre d'espaces.
5. Ecrivez une fonction `compter_autres` qui retourne le nombre de caracteres non alphabetiques.
6. Affichez un recapitulatif complet des statistiques.

Indices

- Utilisez `.chars()` pour obtenir un iterateur sur les caracteres d'une `&str`.
- La methode `.to_lowercase()` convertit un caractere en minuscule.
- Utilisez `match` ou `if` pour tester si un caractere est une voyelle.
- La methode `.is_alphabetic()` teste si un caractere est une lettre.

Correction complete

```
fn main() {  
    // Declaration d'une chaine de caracteres literal (&str immuable)  
    let phrase: &str = "Le langage Rust est fascinant et puissant!";  
  
    println!("Phrase analysee : \"{}\"", phrase);  
    println!("Longueur : {} caracteres\n", phrase.len());  
  
    // Appels des fonctions de comptage  
    let voyelles = compter_voyelles(phrase);  
    let consonnes = compter_consonnes(phrase);  
    let espaces = compter_espaces(phrase);  
    let autres = compter_autres(phrase);  
  
    // Affichage du recapitulatif  
    println!("=== Statistiques ===");  
    println!("Voyelles : {}", voyelles);  
    println!("Consonnes : {}", consonnes);  
    println!("Espaces : {}", espaces);  
}
```

```

println!("Autres caracteres : {}", autres);
println!("Total : {}", voyelles + consonnes + espaces + autres);
}

/// Compte les voyelles dans une chaine de caracteres
/// Parametre :
/// - texte : une reference sur une &str (emprunt immuable)
/// Retourne :
/// - usize : le nombre de voyelles trouvees
fn compter_voyelles(texte: &str) -> usize {
    let mut compteur = 0;

    // Iteration sur chaque caractere de la chaine
    for caractere in texte.chars() {
        // Conversion en minuscule pour le test
        let minuscule = caractere.to_lowercase().next().unwrap();

        // Test si le caractere est une voyelle
        if minuscule == 'a'
            || minuscule == 'e'
            || minuscule == 'i'
            || minuscule == 'o'
            || minuscule == 'u'
            || minuscule == 'y'
        {
            compteur += 1;
        }
    }

    compteur
}

/// Compte les consonnes dans une chaine de caracteres
/// Parametre :
/// - texte : une reference sur une &str
/// Retourne :
/// - usize : le nombre de consonnes
fn compter_consonnes(texte: &str) -> usize {
    let mut compteur = 0;

    for caractere in texte.chars() {
        let minuscule = caractere.to_lowercase().next().unwrap();

        // Une consonne est une lettre qui n'est pas une voyelle
        if minuscule.is_alphabetic()
            && minuscule != 'a'
            && minuscule != 'e'
            && minuscule != 'i'
            && minuscule != 'o'
            && minuscule != 'u'
            && minuscule != 'y'
        {
            compteur += 1;
        }
    }

    compteur
}

```

```

        {
            compteur += 1;
        }
    }

    compteur
}

/// Compte les espaces dans une chaine de caracteres
fn compter_espaces(texte: &str) -> usize {
    let mut compteur = 0;

    for caractere in texte.chars() {
        if caractere == ' ' {
            compteur += 1;
        }
    }

    compteur
}

/// Compte les caracteres non alphabetiques et non espaces
fn compter_autres(texte: &str) -> usize {
    let mut compteur = 0;

    for caractere in texte.chars() {
        if !caractere.is_alphabetic() && caractere != ' ' {
            compteur += 1;
        }
    }

    compteur
}

```

Explication detaillee ligne par ligne

Ligne 3 : `let phrase: &str = "Le langage Rust...";` — Declaration d’une variable de type `&str` (slice de chaine). C’est une **reference** sur une chaine literal stockée en dur dans le binaire. Le type `&str` est une vue empruntée sur une sequence de caracteres UTF-8.

Ligne 7 : `phrase.len()` — Methode qui retourne la longueur en **octets** (pas en caracteres). Cette distinction est importante pour l’Unicode.

Ligne 26 : `fn compter_voyelles(texte: &str) -> usize {` — La fonction prend une **reference** (`&str`) plutot qu’une `String` owned. Cela permet d’emprunter la chaine sans en prendre possession. Le type de retour `usize` est le type entier non signe utilise pour les tailles et les index en Rust.

Ligne 29 : `let mut compteur = 0;` — Le mot-cle `mut` rend la variable mutable (modifiable). Sans `mut`, on ne pourrait pas incrementer `compteur`.

Ligne 32 : `for caractere in texte.chars() {` — La methode `.chars()` retourne un iterateur qui produit chaque caractere Unicode de la chaine un par un. Le type de `caractere` est `char`.

Ligne 34-35 : `let minuscule = caractere.to_lowercase().next().unwrap();` — `.to_lowercase()` retourne un itérateur car la conversion minuscule peut produire plusieurs caractères Unicode (cas complexes). `.next()` prend le premier élément, et `.unwrap()` l'extrait (panique si vide, ce qui n'arrive jamais pour un char valide).

Lignes 38-44 : Le bloc `if` teste si le caractère est une voyelle. Chaque condition est séparée par `||` (OU logique). Si une condition est vraie, le `compteur` est incrémenté.

Ligne 57-64 : La condition pour les consonnes vérifie d'abord que le caractère est alphabétique (`.is_alphabetic()`), puis vérifie qu'il n'est **pas** une voyelle en utilisant `&&` (ET logique) et `!=` (différent de).

Ligne 85 : `if !caractere.is_alphabetic() && caractere != ' '` — Le `!` inverse le booléen. On compte tout ce qui n'est ni alphabétique ni un espace.

Bibliothèques et fonctions utilisées

Fonction	Rôle	Paramètres	Valeur retournée	Exemple supplémentaire
<code>.chars()</code>	Crée un itérateur sur les caractères	Aucun (méthode sur <code>&str</code>)	Chars (itérateur)	<code>"abc".chars()</code> produit <code>'a'</code> , <code>'b'</code> , <code>'c'</code>
<code>.to_lowercase()</code>	Convertit en minuscules	Aucun	ToLowercase (itérateur)	<code>'A'.to_lowercase()</code> donne <code>'a'</code>
<code>.next()</code>	Récupère l'élément suivant de l'itérateur	Aucun	<code>Option<T></code>	<code>Some('a')</code> ou <code>None</code>
<code>.unwrap()</code>	Extrait la valeur d'un <code>Option</code>	Aucun	<code>T</code> (panique si <code>None</code>)	<code>Some(5).unwrap()</code> donne <code>5</code>
<code>.is_alphabetic()</code>	Teste si le caractère est une lettre	Aucun	<code>bool</code>	<code>'3'.is_alphabetic()</code> retourne <code>false</code>
<code>.len()</code>	Retourne la longueur en octets	Aucun	<code>usize</code>	<code>"hello".len()</code> retourne <code>5</code>

Pièges fréquents

Erreur	Message du compilateur	Explication
<code>let compteur = 0;</code> <code>compteur += 1;</code>	<code>cannot assign twice to immutable variable</code>	Oublier <code>mut</code> sur une variable qu' on modifie.
<code>texte: String</code> sans reference	fonctionnel mais moins efficace	La fonction prend possession de la <code>String</code> . Utilisez <code>&str</code> pour emprunter.
<code>phrase.len()</code> pour compter les caracteres	Resultat incorrect pour l' Unicode	<code>.len()</code> compte les octets, pas les caracteres. Utilisez <code>.chars().count()</code> pour les caracteres.

Defi bonus

Ajoutez une fonction `analyser_frequences` qui retourne un `Vec<(char, usize)>` contenant chaque voyelle et son nombre d'occurrences. Par exemple : `[('a', 5), ('e', 8), ...]`.

Exercice 4 : Jeu du Plus ou Moins

Objectifs

- Utiliser la boucle `loop` (boucle infinie)
- Comprendre `break` et `continue`
- Lire l'entree utilisateur avec `std::io::stdin`
- Convertir une `String` en nombre avec `.parse()`
- Gerer les erreurs avec `match` et `Result`

Enonce

Implémentez le celebre jeu du “Plus ou Moins”. Le programme choisit un nombre secret entre 1 et 100, et l'utilisateur doit le deviner. Apres chaque tentative, le programme indique si le nombre secret est plus grand, plus petit, ou egal.

Questions

1. Utilisez un nombre secret fixe (par exemple 42) pour les tests.
2. Lisez l'entree de l'utilisateur avec `std::io::stdin().read_line(...)`.
3. Convertissez l'entree en `u32` avec `.parse()`.
4. Utilisez une boucle `loop` pour permettre plusieurs tentatives.
5. Affichez “Trop grand !”, “Trop petit !” ou “Bravo !” selon la reponse.
6. Comptez et affichez le nombre de tentatives a la fin.

Indices

- `std::io::stdin().read_line(&mut variable)` lit une ligne et la stocke dans `variable`.
- `.parse::<u32>()` convertit une `String` en `u32`. Il retourne un `Result`.
- Utilisez `match` pour gerer le `Result` de `parse`.
- `.trim()` supprime les espaces et retours a la ligne.

Correction complete

```
use std::io;

fn main() {
    // Nombre secret a deviner (fixe pour l'exercice)
    let nombre_secret: u32 = 42;

    println!("=== JEU DU PLUS OU MOINS ===");
    println!("Devinez le nombre entre 1 et 100 !\n");

    // Compteur de tentatives
    let mut tentatives = 0;

    // Boucle infinie qui se termine avec break
    loop {
        println!("Entrez votre proposition :");

        // Declaration d'une String mutable pour stocker l'entree
        let mut proposition = String::new();

        // Lecture de l'entree utilisateur
        io::stdin()
            .read_line(&mut proposition)
            .expect("Erreur de lecture de l'entree");

        // Incrementation du compteur de tentatives
        tentatives += 1;

        // Conversion de la String en u32 avec gestion d'erreur
        let proposition: u32 = match proposition.trim().parse() {
            Ok(nombre) => nombre,
            Err(_) => {
                println!("Ce n'est pas un nombre valide !\n");
                continue; // Passe a l'iteration suivante
            }
        };

        // Comparaison avec le nombre secret
        if proposition < nombre_secret {
            println!("Trop petit !\n");
        } else if proposition > nombre_secret {
            println!("Trop grand !\n");
        } else {
            println!("\n*** Bravo ! Le nombre etait {} ! ***", nombre_secret);
            println!("Nombre de tentatives : {}", tentatives);
            break; // Sortie de la boucle
        }
    }
}
```

Explication detaillee ligne par ligne

Ligne 1 : `use std::io;` — Import du module `io` de la bibliotheque standard. Ce module fournit les fonctions d'entree/sortie, notamment `stdin()` pour lire l'entree clavier.

Ligne 4 : `let nombre_secret: u32 = 42;` — Declaration du nombre secret. Le type `u32` (entier non signe 32 bits) est choisi car on travaille avec des nombres positifs entre 1 et 100.

Ligne 10 : `let mut tentatives = 0;` — Variable mutable pour compter les tentatives. `mut` est necessaire car on va l'incrementer.

Ligne 13 : `loop {` — Debut d'une boucle infinie. Cette boucle ne s'arretera que lorsqu'on rencontrera un `break`.

Ligne 16 : `let mut proposition = String::new();` — Creation d'une nouvelle `String` vide et mutable. A chaque iteration, on recree cette variable pour stocker la nouvelle proposition.

Ligne 19-21 : `io::stdin().read_line(&mut proposition)` — Appel de `stdin()` pour obtenir le handle d'entree standard, puis `read_line` pour lire une ligne. La methode prend une **reference mutable** (`&mut`) sur notre `String` car elle va la modifier en y ecrivant les caracteres tapes.

Ligne 22 : `.expect("Erreur de lecture");` — La methode `read_line` retourne un `Result`. `.expect()` extrait la valeur en cas de succes, ou panique avec le message donne en cas d'erreur.

Ligne 28-33 : `match proposition.trim().parse() { ... }` — Chaine de traitement :

- `.trim()` supprime les espaces blancs de debut et fin, y compris le `\n` (retour a la ligne) que `read_line` inclut.
- `.parse()` tente de convertir la chaine en un nombre. Le type `u32` est inferé par le compilateur grace a l'annotation de la ligne suivante.
- `match` gere les deux cas du `Result` : `Ok(nombre)` en cas de succes, `Err(_)` en cas d'echec (le `_` ignore la valeur d'erreur).

Ligne 31 : `continue;` — Passe immediatement a l'iteration suivante de la boucle. Si la conversion echoue, on ne fait pas la comparaison.

Lignes 36-44 : Bloc conditionnel `if/else if/else` qui compare la proposition au nombre secret.

Ligne 41 : `break;` — Sort definitivement de la boucle `loop`. Le programme se termine apres l'affichage du message de felicitations.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>std::io::stdin()</code>	Obtient l' entree standard	Aucun	<code>Stdin</code>	-
<code>.read_line(&mut buf)</code>	Lit une ligne dans le buffer	<code>&mut String</code>	<code>Result<usize></code>	Renvoie le nombre d' octets lus

Table 9 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.trim()</code>	Supprime les espaces blancs	Aucun	<code>&str</code>	<code>" hello ".trim() -> "hello"</code>
<code>.parse()</code>	Convertit une chaine en type cible	Aucun (type inferé)	<code>Result<T, E></code>	<code>"42".parse::<i32>() -> Ok(42)</code>
<code>loop { }</code>	Boucle infinie	Aucun	Peut retourner une valeur	<code>let x = loop { break 5; };</code>
<code>break</code>	Sort de la boucle	Optionnellement une valeur	Sort de la boucle	-
<code>continue</code>	Passe a l'iteration suivante	Aucun	-	-

Pieges frequents

Erreur	Message du compilateur	Explication
Oublier <code>.trim()</code>	Parse echoue car <code>"42\\n"</code> n' est pas un nombre	<code>read_line</code> inclut le retour a la ligne.
<code>proposition: String</code> sans <code>mut</code>	<code>cannot borrow as mutable</code>	<code>read_line</code> necessite une reference mutable.
Oublier tentatives <code>+= 1;</code>	Variable non mutable	Sans <code>mut</code> , impossible de modifier.
<code>parse()</code> sans <code>match</code>	<code>Result</code> non utilise	Le compilateur warn si un <code>Result</code> est ignore.

Defi bonus

Generez le nombre secret aleatoirement avec la crate `rand`. Ajoutez un systeme de score base sur le nombre de tentatives, et demandez a l'utilisateur s'il veut rejouer.

Exercice 5 : Table de Multiplication

Objectifs

- Utiliser les boucles `for` imbriquees
- Comprendre les ranges (`..` et `..=`)
- Formater l'affichage avec des largeurs fixes
- Ecrire des fonctions parametrees

Enonce

Creez un programme qui affiche une table de multiplication de taille variable. L'utilisateur specifie la taille (par exemple 10 pour une table 10x10).

Questions

1. Ecrivez une fonction `afficher_table` qui prend un parametre `taille: usize`.
2. Affichez la ligne d'en-tete avec les nombres de 1 a `taille`.
3. Utilisez deux boucles `for` imbriquees pour generer la table.
4. Formatez l'affichage pour que chaque nombre occupe 4 caracteres minimum.
5. Affichez le separateur entre l'en-tete et le contenu.
6. Dans le `main`, testez avec `taille = 10` puis `taille = 5`.

Indices

- La syntaxe `1..=n` cree un range inclusif (de 1 a n).
- La syntaxe `1..n` cree un range exclusif (s'arrete a n-1).
- `{:>4}` dans `println!` aligne a droite sur 4 caracteres.
- `{:-<20}` cree une ligne de 20 tirets.

Correction complete

```
fn main() {
    println!("Table de multiplication 10x10 :\n");
    afficher_table(10);

    println!("\nTable de multiplication 5x5 :\n");
    afficher_table(5);
}

/// Affiche une table de multiplication de taille N x N
/// Parametre :
/// - taille : usize determinant la dimension de la table
fn afficher_table(taille: usize) {
    // Affichage de la ligne d'en-tete
    print!("{:>4} |", ""); // Coin vide superieur gauche
    for col in 1..=taille {
        print!("{:>4}", col);
    }
    println(); // Retour a la ligne

    // Ligne de separation
    print!("{:-<4}-+", "");
    for _ in 1..=taille {
        print!("{:-<5}", "");
    }
    println();

    // Corps de la table : boucles imbriquees
    for ligne in 1..=taille {
        // Numero de ligne
```

```
print!("{:>4} |", ligne);

// Calcul et affichage des produits
for col in 1..=taille {
    let produit = ligne * col;
    print!(" {:>4}", produit);
}

println(); // Retour a la ligne apres chaque ligne
}
```

Explication detaillee ligne par ligne

Ligne 13 : `fn afficher_table(taille: usize) {` — La fonction prend un `usize` (entier non signe pour les tailles/index). Ce type est standard pour les dimensions en Rust.

Ligne 15-16 : `print!("{:>4} |", "");` — Affiche le coin superieur gauche vide. `{:>4}` reserve 4 caracteres et aligne a droite. Le `print!` (sans `ln`) n’ajoute pas de retour a la ligne.

Lignes 17-19 : Boucle `for col in 1..=taille { ... }` — Le range `1..=taille` est **inclusif** : il va de 1 jusqu’a `taille` inclusivement. Le `=` apres `..` rend la borne inclusive.

Lignes 22-25 : Ligne de separation. `{:-<4}` cree une chaine de 4 tirets alignes a gauche (le `-` est le caractere de remplissage). Le `+` marque l’intersection.

Ligne 28 : `for ligne in 1..=taille {` — Boucle exterieure qui parcourt les lignes.

Ligne 33 : `for col in 1..=taille {` — Boucle interieure qui parcourt les colonnes. Cette boucle s’execute completement pour chaque iteration de la boucle exterieure.

Ligne 34 : `let produit = ligne * col;` — Multiplication des deux index pour obtenir le produit.

Ligne 35 : `print!(" {:>4}", produit);` — Affiche le produit avec un espace initial et 4 caracteres de largeur, aligne a droite.

Ligne 38 : `println();` — Retour a la ligne apres avoir affiche toutes les colonnes d’une ligne.

Bibliotheques et fonctions utilisees

Fonction/Macro	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>print!</code>	Affiche sans retour a la ligne	Format + arguments	<code>()</code>	<code>print!("{}", x);</code>
<code>println!</code>	Affiche avec retour a la ligne	Format + arguments	<code>()</code>	<code>println();</code> pour un saut de ligne
<code>1..=n</code>	Range inclusif	Borne de fin	<code>RangeInclusive</code>	<code>for i in 1..=5 -> 1,2,3,4,5</code>

Table 11 – continued

Fonction/Macro	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>1..n</code>	Range exclusif	Borne de fin	Range	<code>for i in 1..5 -> 1,2,3,4</code>
<code>{:>4}</code>	Format aligne a droite, 4 chars	Valeur	Formatée	<code>println!("{:>6}", 42);</code> affiche " 42"
<code>{:-<5}</code>	Format aligne a gauche, 5 tirets	Valeur	Formatée	<code>println!("{:*<10}", "");</code> affiche "*****"

Pieges frequents

Erreur	Message du compilateur	Explication
<code>1..taille</code> au lieu de <code>1..=taille</code>	Table incomplete (manque la derniere ligne/colonne)	Le range <code>..</code> est exclusif. Utilisez <code>..=</code> pour inclusif.
<code>println!</code> au lieu de <code>print!</code> dans la boucle	Table mal formatee avec des sauts de ligne parasites	Utilisez <code>print!</code> pour rester sur la meme ligne.
<code>taille: i32</code>	<code>type mismatch</code> dans les index	Les index de tableaux/vecteurs utilisent <code>usize</code> .

Defi bonus

Ajoutez une fonction `afficher_table_limitee` qui prend deux parametres `debut` et `fin` et n'affiche que la sous-table correspondante (par exemple de 5 a 10 uniquement).

Exercices 6 a 10 : Intermediaire

Exercice 6 : Manipulation de Vecteurs — Nombres Premiers

Objectifs

- Creer et manipuler des `Vec<T>`
- Utiliser `.push()` pour ajouter des elements
- Iterer sur un vecteur avec `.iter()`
- Comprendre la propriete et l'emprunt avec les vecteurs

Enonce

Ecrivez un programme qui trouve tous les nombres premiers jusqu'a une limite donnee en utilisant le crible d'Eratosthene, puis qui les affiche.

Questions

1. Créez une fonction `crible_eratosthene` qui prend `limite: usize` et retourne `Vec<usize>`.
2. Créez un vecteur `nombres_premiers` vide.
3. Créez un vecteur booléen `est_premier` de taille `limite + 1`, initialise à `true`.
4. Marquez 0 et 1 comme non-premiers.
5. Implémentez l'algorithme du crible : pour chaque nombre premier trouvé, marquez ses multiples.
6. Collectez les nombres restants marqués `true` dans `nombres_premiers`.
7. Affichez les nombres premiers trouvés et leur quantité.

Indices

- `vec![true; n]` crée un vecteur de `n` éléments tous initialisés à `true`.
- `.push(element)` ajoute un élément à la fin du vecteur.
- Utilisez `n * n <= limite` comme condition d'arrêt pour le crible.
- Les multiples d'un nombre `p` commencent à `p * p`.

Correction complete

```
fn main() {
    let limite: usize = 100;

    println!("Nombres premiers jusqu'à {} :\n", limite);

    // Appel de la fonction qui retourne un Vec<usize>
    let premiers = crible_eratosthene(limite);

    // Affichage des nombres premiers
    println!("Trouvés : {} nombres premiers", premiers.len());

    // Affichage formate : 10 nombres par ligne
    for (index, &nombre) in premiers.iter().enumerate() {
        print!("{:>4}", nombre);

        // Retour à la ligne tous les 10 nombres
        if (index + 1) % 10 == 0 {
            println();
        }
    }

    // Retour à la ligne final si nécessaire
    if premiers.len() % 10 != 0 {
        println();
    }
}

/// Implementation du crible d'Eratosthene
/// Parametre :
/// - limite : la borne superieure de recherche
/// Retourne :
/// - Vec<usize> contenant tous les nombres premiers <= limite
```

```

fn crible_eratosthene(limite: usize) -> Vec<usize> {
    // Cas particulier : limite < 2
    if limite < 2 {
        return Vec::new(); // Retourne un vecteur vide
    }

    // Vecteur de booleans : true = potentiellement premier
    let mut est_premier: Vec<bool> = vec![true; limite + 1];

    // 0 et 1 ne sont pas premiers
    est_premier[0] = false;
    est_premier[1] = false;

    // Crible : pour chaque nombre de 2 a sqrt(limite)
    let mut nombre = 2;
    while nombre * nombre <= limite {
        // Si nombre est marque comme premier
        if est_premier[nombre] {
            // Marquer tous les multiples comme non-premiers
            let mut multiple = nombre * nombre;
            while multiple <= limite {
                est_premier[multiple] = false;
                multiple += nombre;
            }
        }
        nombre += 1;
    }

    // Collecter les nombres marques comme premiers
    let mut nombres_premiers: Vec<usize> = Vec::new();

    for n in 2..=limite {
        if est_premier[n] {
            nombres_premiers.push(n);
        }
    }

    nombres_premiers
}

```

Explication detaillee ligne par ligne

Ligne 7 : `let premiers = crible_eratosthene(limite);` — La fonction retourne un `Vec<usize>`. La **propriete** du vecteur est transferee a la variable `premiers` dans `main`. Le vecteur est **owned** par `premiers`.

Ligne 13 : `for (index, &nombre) in premiers.iter().enumerate() {` — Combine plusieurs operations :

- `.iter()` cree un iterateur qui **emprunte** les elements du vecteur (reference immuable)
- `.enumerate()` ajoute l'index a chaque element
- `&nombre` utilise le **pattern matching** pour dereferencer, obtenant directement la valeur `usize`

Ligne 37 : `let mut est_premier: Vec<bool> = vec![true; limite + 1];` — Macro `vec!` qui cree un vecteur de taille `limite + 1` avec tous les elements initialises a `true`. Le `+ 1` est necessaire car on veut un index pour la valeur `limite` elle-meme.

Lignes 40-41 : `est_premier[0] = false;` — Acces a un element du vecteur par index. Le vecteur doit etre mutable (`mut`) pour modifier ses elements.

Ligne 45 : `while nombre * nombre <= limite {` — Condition d'arret optimisee. On n'a besoin de cribler que jusqu'a la racine carree de la limite.

Lignes 49-53 : Boucle interne qui marque les multiples. On commence a `nombre * nombre` car les multiples inferieurs ont deja ete marques par des nombres plus petits.

Ligne 60 : `let mut nombres_premiers: Vec<usize> = Vec::new();` — Creation d'un vecteur vide. `Vec::new()` est un constructeur associe (pas besoin d'un type concret pour l'appeler).

Ligne 64 : `nombres_premiers.push(n);` — Ajout d'un element a la fin du vecteur. Le vecteur gere automatiquement sa capacite.

Ligne 68 : `nombres_premiers` — Expression de retour (derniere expression, sans point-virgule). La propriete du vecteur est transferee a l'appelant.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>vec![val; n]</code>	Cree un vecteur avec n fois val	Valeur, nombre	<code>Vec<T></code>	<code>vec![0; 5] -> [0,0,0,0,0]</code>
<code>Vec::new()</code>	Cree un vecteur vide	Aucun	<code>Vec<T></code>	<code>let v: Vec<i32> = Vec::new();</code>
<code>.push(elem)</code>	Ajoute un element a la fin	Element a ajouter	<code>()</code>	<code>v.push(42);</code>
<code>.len()</code>	Nombre d'elements	Aucun	<code>usize</code>	<code>vec![1,2,3].len() -> 3</code>
<code>.iter()</code>	Iterateur d'emprunt (references)	Aucun	<code>slice::Iter<T></code>	<code>for x in v.iter()</code>
<code>.enumerate()</code>	Ajoute l'index a l'iterateur	Aucun	<code>Iterator<Item=(usize, &T)></code>	<code>(0, &1), (1, &2)...</code>

Pieges frequents

Erreur	Message du compilateur	Explication
<code>est_premier[nombre]</code> cannot borrow as mutable sans <code>mut</code>		Le vecteur doit etre declare <code>mut</code> pour modifier ses elements.
<code>vec![true; limite]</code> (sans <code>+1</code>)	Index hors bornes pour <code>est_premier[limite]</code>	Le vecteur a <code>limite</code> elements (index 0 a <code>limite-1</code>). Il faut <code>limite + 1</code> .
<code>premiers</code> utilise apres <code>premiers.iter()</code>	Potentiel conflit d' emprunt	<code>.iter()</code> emprunte le vecteur. L' emprunt se termine a la fin de la boucle.
Oublier le <code>return</code> pour le cas <code>< 2</code>	Le compilateur retourne la derniere expression	Utilisez <code>return Vec::new();</code> ou structurez le code pour que <code>nombres_premiers</code> soit la derniere expression.

Defi bonus

Ajoutez une fonction `decomposition_premiers` qui prend un nombre et retourne un `Vec<(usize, usize)>` contenant les facteurs premiers et leur multiplicite (par exemple, $60 = 2^2 * 3 * 5 \rightarrow [(2,2), (3,1), (5,1)]$).

Exercice 7 : Traitement de Texte avec String

Objectifs

- Maitriser `String` vs `&str`
- Comprendre la propriete et le clonage des `String`
- Utiliser `.push()`, `.push_str()`, `.clone()`
- Manipuler les methodes de traitement de texte

Enonce

Creez un programme qui manipule des chaines de caracteres : inversion, concatenation, comptage de mots, et remplacement de caracteres.

Questions

1. Ecrivez une fonction `inverser_chaine` qui prend une `&str` et retourne une `String` inversee.
2. Ecrivez une fonction `concatener` qui prend deux `&str` et retourne une `String`.
3. Ecrivez une fonction `compter_mots` qui compte le nombre de mots dans une phrase.
4. Ecrivez une fonction `remplacer_voyelles` qui remplace toutes les voyelles par `*`.
5. Ecrivez une fonction `est_palindrome` qui verifie si un mot est un palindrome.
6. Testez toutes ces fonctions dans le `main`.

Indices

- `.chars()` donne un iterateur sur les caracteres.
- `.rev()` inverse un iterateur.
- `.collect::<String>()` rassemble les caracteres en une `String`.

- `.split_whitespace()` separe une phrase en mots.
- `.to_string()` convertit `&str` en `String`.

Correction complete

```
fn main() {
    let texte: &str = "Le langage Rust est puissant";

    println!("Texte original : \"{}\"", texte);

    // 1. Inversion de chaine
    let inverse = inverser_chaine(texte);
    println!("Inverse : \"{}\"", inverse);

    // 2. Concatenation
    let texte2: &str = " et elegant";
    let concatene = concatener(texte, texte2);
    println!("Concatene : \"{}\"", concatene);

    // 3. Comptage de mots
    let nb_mots = compter_mots(texte);
    println!("Nombre de mots : {}", nb_mots);

    // 4. Remplacement des voyelles
    let sans_voyelles = remplacer_voyelles(texte);
    println!("Sans voyelles : \"{}\"", sans_voyelles);

    // 5. Test de palindrome
    let mot1 = "radar";
    let mot2 = "hello";
    println!("\"{}\" est palindrome : {}", mot1, est_palindrome(mot1));
    println!("\"{}\" est palindrome : {}", mot2, est_palindrome(mot2));
}

/// Inverse une chaine de caracteres
/// Parametre :
///   - s : &str a inverser
/// Retourne :
///   - String contenant les caracteres dans l'ordre inverse
fn inverser_chaine(s: &str) -> String {
    s.chars().rev().collect()
}

/// Concatene deux chaines
/// Parametres :
///   - s1, s2 : les deux &str a concatener
/// Retourne :
///   - String resultat de la concatenation
fn concatener(s1: &str, s2: &str) -> String {
    let mut resultat = String::new();
    resultat.push_str(s1);
```



```

    resultat.push_str(s2);
    resultat
}

/// Compte le nombre de mots dans une phrase
/// Un mot est defini comme une sequence separee par des espaces
/// Parametre :
///   - s : &str contenant la phrase
/// Retourne :
///   - usize : le nombre de mots
fn compter_mots(s: &str) -> usize {
    s.split_whitespace().count()
}

/// Remplace toutes les voyelles par le caractere '*'
/// Parametre :
///   - s : &str a traiter
/// Retourne :
///   - String avec les voyelles remplacees
fn remplacer_voyelles(s: &str) -> String {
    let mut resultat = String::new();

    for caractere in s.chars() {
        let minuscule = caractere.to_lowercase().next().unwrap();

        if minuscule == 'a'
           || minuscule == 'e'
           || minuscule == 'i'
           || minuscule == 'o'
           || minuscule == 'u'
           || minuscule == 'y'
        {
            resultat.push('*');
        } else {
            resultat.push(caractere);
        }
    }

    resultat
}

/// Verifie si un mot est un palindrome
/// Un palindrome se lit identiquement dans les deux sens
/// Parametre :
///   - s : &str a verifier
/// Retourne :
///   - true si palindrome, false sinon
fn est_palindrome(s: &str) -> bool {
    let inverse = inverser_chaine(s);
    s == inverse
}

```

```
}
```

Explication detaillee ligne par ligne

Ligne 28 : `fn inverser_chaine(s: &str) -> String {` — La fonction prend une **reference** (`&str`) pour emprunter la chaine sans la posseder. Elle retourne une **String owned** car la chaine inversee est une nouvelle allocation.

Ligne 29 : `s.chars().rev().collect()` — Chaine de methodes :

- `s.chars()` : iterateur sur les caracteres (emprunt immuable)
- `.rev()` : inverse l'ordre de l'iterateur
- `.collect()` : consomme l'iterateur et rassemble les elements. Le type `String` est inferé grace au type de retour de la fonction.

Lignes 37-41 : `concatener` construit une `String` manuellement :

- `String::new()` cree une `String` vide
- `.push_str(s1)` ajoute tout le contenu de `s1`
- `.push_str(s2)` ajoute tout le contenu de `s2`
- `resultat` est retournee (propriete transferee)

Ligne 49 : `s.split_whitespace().count()` — `.split_whitespace()` retourne un iterateur qui produit les “mots” (separés par tout caractere d’espace). `.count()` consomme l’iterateur et compte les elements.

Lignes 57-73 : `remplacer_voyelles` construit une nouvelle `String` caractere par caractere. On utilise `.push(' ')` pour ajouter un caractere individuel, et `.push(caractere)` pour conserver le caractere original.

Ligne 84-85 : `est_palindrome` compare la chaine originale avec son inverse. Cette comparaison fonctionne entre `&str` et `String` car Rust compare le contenu, pas le type exact.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.chars()</code>	Iterateur sur les caracteres	Aucun	Chars	<code>"abc".chars() -> 'a', 'b', 'c'</code>
<code>.rev()</code>	Inverse un iterateur	Aucun	<code>Rev<Iter></code>	<code>[1,2,3].iter().rev() -> 3,2,1</code>
<code>.collect()</code>	Rassemble un iterateur en collection	Aucun (type inferé)	<code>Collection<T></code>	<code>.collect::<Vec<_>>()</code>
<code>.push(c)</code>	Ajoute un caractere	<code>char</code>	<code>()</code>	<code>s.push('!');</code>

Table 15 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.push_str(s)</code>	Ajoute une chaine	<code>&str</code>	<code>()</code>	<code>s.push_str("world");</code>
<code>.split_whitespace()</code>	Separe par espaces	Aucun	<code>SplitWhitespace</code>	<code>"a b".split_whitespace()</code>
<code>.count()</code>	Compte les elements	Aucun	<code>usize</code>	<code>[1,2].iter().count() -> 2</code>
<code>.to_string()</code>	Convertit en String	Aucun	<code>String</code>	<code>"hello".to_string()</code>
<code>String::new()</code>	Cree une String vide	Aucun	<code>String</code>	<code>let s = String::new();</code>

Pieges frequents

Erreur	Message du compilateur	Explication
<code>s == inverse</code> avec <code>s: String</code> et <code>inverse: &str</code>	Fonctionne par deref coercion	Rust compare le contenu. Mais attention aux types !
<code>resultat.push(s1)</code>	<code>expected char, found &str</code>	<code>.push()</code> prend un <code>char</code> , <code>.push_str()</code> prend une <code>&str</code> .
Oublier <code>.collect()</code>	Iterator n' est pas String	L' iterateur doit etre rassemble en String.
<code>s.chars().rev()</code> sans collect	-	L' iterateur est lazy : il ne fait rien tant qu' on ne le consomme pas.

Defi bonus

Ajoutez une fonction `compression_rle` qui implemente le Run-Length Encoding : "aaabbc" devient "a3b2c1". La fonction doit prendre une `&str` et retourner une `String`.

Exercice 8 : Filtrage et Transformation de Vec

Objectifs

- Utiliser les iterateurs et leurs adaptateurs
- Comprendre `.iter()`, `.map()`, `.filter()`, `.collect()`
- Chainer les operations d'iterateurs
- Distinguer les closures et les fonctions nommees

Enonce

Manipulez un vecteur de nombres entiers en appliquant diverses transformations fonctionnelles.

Questions

1. Creez un vecteur `nombres` contenant les valeurs de 1 a 20.
2. Ecrivez une fonction `filtrer_pairs` qui retourne seulement les nombres pairs.
3. Ecrivez une fonction `doubler` qui multiplie chaque element par 2.
4. Ecrivez une fonction `somme` qui calcule la somme de tous les elements.
5. Ecrivez une fonction `trouver_plus_grand` qui retourne le maximum.
6. Dans le `main`, chainez ces operations : filtrez les pairs, doublez-les, puis calculez la somme.

Indices

- `.filter(|x| x % 2 == 0)` filtre les nombres pairs.
- `.map(|x| x * 2)` transforme chaque element.
- `.sum()` calcule la somme (le type doit etre inferable).
- `.max()` retourne un `Option<T>`.

Correction complete

```
fn main() {  
    // Creation d'un vecteur de 1 a 20  
    let nombres: Vec<i32> = (1..=20).collect();  
    println!("Vecteur original : {:?}\n", nombres);  
  
    // 1. Filtrer les nombres pairs  
    let pairs = filtrer_pairs(&nombres);  
    println!("Nombres pairs : {:?}", pairs);  
  
    // 2. Doubler chaque element  
    let doubles = doubler(&nombres);  
    println!("Nombres doubles : {:?}", doubles);  
  
    // 3. Calculer la somme  
    let total = somme(&nombres);  
    println!("Somme totale : {}", total);  
  
    // 4. Trouver le maximum  
    match trouver_plus_grand(&nombres) {  
        Some(max) => println!("Maximum : {}", max),  
        None => println!("Vecteur vide"),  
    }  
  
    // 5. Chainer les operations : pairs -> doubler -> somme  
    let resultat: i32 = nombres  
        .iter()  
        .filter(|&x| x % 2 == 0) // Filtrer les pairs  
        .map(|&x| x * 2)        // Doubler  
        .sum();                 // Sommer
```

```

println!("\nChaine (pairs doubles, somme) : {}", resultat);

// Version alternative avec collect() intermediaire
let pairs_doubles: Vec<i32> = nombres
    .iter()
    .filter(|&&x| x % 2 == 0)
    .map(|&x| x * 2)
    .collect();

println!("Vecteur pairs doubles : {:?}", pairs_doubles);
}

/// Filtre les nombres pairs d'un vecteur
/// Parametre :
/// - v : reference sur Vec<i32>
/// Retourne :
/// - Vec<i32> contenant uniquement les nombres pairs
fn filtrer_paires(v: &Vec<i32>) -> Vec<i32> {
    v.iter()
        .filter(|&&x| x % 2 == 0)
        .copied()
        .collect()
}

/// Double chaque element d'un vecteur
/// Parametre :
/// - v : reference sur Vec<i32>
/// Retourne :
/// - Vec<i32> avec chaque element multiplie par 2
fn doubler(v: &Vec<i32>) -> Vec<i32> {
    v.iter()
        .map(|&x| x * 2)
        .collect()
}

/// Calcule la somme des elements
/// Parametre :
/// - v : reference sur Vec<i32>
/// Retourne :
/// - i32 : la somme de tous les elements
fn somme(v: &Vec<i32>) -> i32 {
    v.iter().sum()
}

/// Trouve le maximum dans un vecteur
/// Parametre :
/// - v : reference sur Vec<i32>
/// Retourne :
/// - Option<i32> : Some(&max) ou None si vide
fn trouver_plus_grand(v: &Vec<i32>) -> Option<i32> {
    v.iter().max()
}

```

```
}
```

Explication detaillee ligne par ligne

Ligne 4 : `let nombres: Vec<i32> = (1..=20).collect();` — Le range `1..=20` cree un iterateur de 1 a 20. `.collect()` rassemble ces nombres dans un `Vec<i32>`. L’annotation de type `Vec<i32>` est necessaire car `.collect()` peut produire plusieurs types de collections.

Ligne 27 : `match trouver_plus_grand(&nombres) { ... }` — `.max()` retourne un `Option<&i32>` car le vecteur pourrait etre vide. Le `match` gere les deux cas.

Lignes 33-38 : Chaine d’iterateurs :

- `.iter()` : emprunte le vecteur, produit des `&i32`
- `.filter(|&&x| x % 2 == 0)` : closure qui filtre. `|&&x|` utilise le pattern matching pour dereferencer deux niveaux : l’iterateur produit `&&i32` (reference a travers le filter).
- `.map(|&x| x * 2)` : dereference puis multiplication
- `.sum()` : consomme l’iterateur et calcule la somme

Lignes 55-58 : `v.iter().filter(|&&x| x % 2 == 0).copied().collect()` — La methode `.copied()` copie les valeurs referencees (`&i32 -> i32`). C’est un raccourci pour `.map(|&x| x)`. Les `i32` implementent le trait `Copy`, donc cette operation est bon marche.

Ligne 72 : `v.iter().sum()` — `.sum()` consomme l’iterateur. Le type de retour est determine par le contexte (ici `i32` grace a la signature de la fonction).

Ligne 81 : `v.iter().max()` — Retourne `Option<&i32>`. Si le vecteur est vide, retourne `None`.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.iter()</code>	Iterateur d'emprunt	Aucun	<code>slice::Iter<T></code>	Produit <code>&T</code>
<code>.filter(closure)</code>	Filtre les elements	Closure <code>FnMut(&T) -> bool</code>	<code>Filter</code> (iterateur)	Lazy evaluation
<code>.map(closure)</code>	Transforme chaque element	Closure <code>FnMut(T) -> U</code>	<code>Map</code> (iterateur)	Lazy evaluation
<code>.collect()</code>	Rassemble en collection	Aucun	Type inferé	<code>.collect::<Vec<_>>()</code>
<code>.sum()</code>	Calcule la somme	Aucun	Type inferé	<code>[1,2].iter().sum::<i32>()</code>
<code>.max()</code>	Trouve le maximum	Aucun	<code>Option<&T></code>	Retourne <code>None</code> si vide

Table 17 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.copied()</code>	Copie les elements references	Aucun	<code>Copied<Iter></code>	Necessite que T implemente Copy
<code>(a..=b)</code>	Range inclusif	Bornes	<code>RangeInclusive</code>	<code>1..=5 -> 1,2,3,4,5</code>

Pieges frequents

Erreur	Message du compilateur	Explication
<code> x x % 2 == 0</code> dans filter	<code>cannot apply modulo to &i32</code>	Le filter recoit <code>&&i32</code> . Il faut <code> &&x </code> ou <code> x **x % 2 == 0</code> .
Oublier l'annotation de type sur collect	<code>cannot infer type</code>	Rust ne sait pas quel type de collection creer.
<code>.sum()</code> sans type inferable	<code>cannot infer type</code>	Precisez avec <code>.sum::<i32>()</code> .
<code>v.iter().max()</code> dereference direct	Type mismatch	<code>.max()</code> retourne <code>Option<&i32></code> , pas <code>Option<i32></code> .

Defi bonus

Implementez une fonction `median` qui calcule la mediane d'un `Vec<i32>`. La mediane est la valeur centrale apres tri (ou la moyenne des deux valeurs centrales si le nombre d'elements est pair).

Exercice 9 : Recherche dans un Vec

Objectifs

- Implementer des algorithmes de recherche
- Comprendre `Option<T>` en profondeur
- Manipuler les references et l'emprunt
- Utiliser les patterns `match` et `if let`

Enonce

Implementez differentes fonctions de recherche dans un vecteur d'entiers : recherche lineaire, recherche du premier/minimum, et recherche de tous les indices correspondants.

Questions

1. Ecrivez `recherche_lineaire` qui retourne `Option<usize>` (l'index trouve).
2. Ecrivez `trouver_minimum` qui retourne `Option<&i32>`.

3. Ecrivez `trouver_tous_indices` qui retourne `Vec<usize>` de tous les indices ou une valeur apparait.
4. Ecrivez `compter_occurrences` qui compte combien de fois une valeur apparait.
5. Ecrivez `est_trie` qui verifie si le vecteur est trie en ordre croissant.
6. Testez toutes ces fonctions dans le `main` avec un vecteur de donnees.

Indices

- `Option<usize> : Some(index)` si trouve, `None` sinon.
- `.enumerate()` donne l'index et la valeur lors d'une iteration.
- Pour le minimum, initialisez avec le premier element.
- Pour `est_trie`, comparez chaque element avec le suivant.

Correction complete

```
fn main() {
    let donnees: Vec<i32> = vec![34, 12, 56, 12, 89, 12, 7, 45];
    let cible: i32 = 12;

    println!("Donnees : {:?}\n", donnees);

    // 1. Recherche lineaire
    match recherche_lineaire(&donnees, cible) {
        Some(index) => println!("{}", trouve a l'index {} ", cible, index),
        None => println!("{}", non trouve ", cible),
    }

    // 2. Trouver le minimum
    match trouver_minimum(&donnees) {
        Some(min) => println!("Minimum : {} ", min),
        None => println!("Vecteur vide "),
    }

    // 3. Trouver tous les indices
    let indices = trouver_tous_indices(&donnees, cible);
    println!("Indices de {} : {:?}", cible, indices);

    // 4. Compter les occurrences
    let compte = compter_occurrences(&donnees, cible);
    println!("Occurrences de {} : {} ", cible, compte);

    // 5. Verifier si trie
    let trie_croissant: Vec<i32> = vec![1, 2, 3, 4, 5];
    println!("Donnees trierees : {} ", est_trie(&donnees));
    println!("trie_croissant trie : {} ", est_trie(&trie_croissant));
}

/// Recherche lineaire dans un vecteur
/// Parametres :
///   - v : reference sur le vecteur a chercher
///   - cible : la valeur recherchee
/// Retourne :
```



```

/// - Option<usize> : Some(index) ou None
fn recherche_lineaire(v: &Vec<i32>, cible: i32) -> Option<usize> {
    for (index, &valeur) in v.iter().enumerate() {
        if valeur == cible {
            return Some(index);
        }
    }
    None
}

/// Trouve le minimum dans un vecteur
/// Parametre :
/// - v : reference sur le vecteur
/// Retourne :
/// - Option<i32> : reference sur le minimum, ou None
fn trouver_minimum(v: &Vec<i32>) -> Option<i32> {
    if v.is_empty() {
        return None;
    }

    let mut min = &v[0];

    for valeur in v.iter() {
        if valeur < min {
            min = valeur;
        }
    }

    Some(min)
}

/// Trouve tous les indices d'une valeur
/// Parametres :
/// - v : reference sur le vecteur
/// - cible : la valeur recherchee
/// Retourne :
/// - Vec<usize> : vecteur de tous les indices trouves
fn trouver_tous_indices(v: &Vec<i32>, cible: i32) -> Vec<usize> {
    let mut indices: Vec<usize> = Vec::new();

    for (index, &valeur) in v.iter().enumerate() {
        if valeur == cible {
            indices.push(index);
        }
    }

    indices
}

/// Compte les occurrences d'une valeur
/// Parametres :

```

```

/// - v : reference sur le vecteur
/// - cible : la valeur a compter
/// Retourne :
/// - usize : nombre d'occurrences
fn compter_occurrences(v: &Vec<i32>, cible: i32) -> usize {
    let mut compteur = 0;

    for &valeur in v.iter() {
        if valeur == cible {
            compteur += 1;
        }
    }

    compteur
}

/// Verifie si un vecteur est trie en ordre croissant
/// Parametre :
/// - v : reference sur le vecteur
/// Retourne :
/// - true si trie en ordre croissant, false sinon
fn est_trie(v: &Vec<i32>) -> bool {
    if v.len() < 2 {
        return true; // Un vecteur vide ou a 1 element est toujours trie
    }

    for i in 0..(v.len() - 1) {
        if v[i] > v[i + 1] {
            return false;
        }
    }

    true
}

```

Explication detaillee ligne par ligne

Ligne 30 : `for (index, &valeur) in v.iter().enumerate() {` — `.enumerate()` produit des tuples (`usize`, `&i32`). Le pattern `(index, &valeur)` destructure le tuple et dereference la valeur en meme temps.

Ligne 31 : `if valeur == cible {` — Comparaison directe car `valeur` est un `i32` (copiee via le pattern `&valeur`).

Ligne 32 : `return Some(index);` — Retour anticipatif avec `return`. Des qu'on trouve la valeur, on sort de la fonction.

Ligne 34 : `None` — Si la boucle termine sans trouver la valeur, on retourne `None`. C'est la derniere expression de la fonction.

Ligne 43 : `let mut min = &v[0];` — `min` est une **reference** sur un element du vecteur. Elle pointe vers `v[0]` initialement. Le type de `min` est `&i32`.

Ligne 45-48 : `for valeur in v.iter() { ... }` — Ici `valeur` est de type `&i32` (reference). La

comparaison `valeur < min` fonctionne car Rust compare les valeurs pointees (deref coercion implicite pour les comparaisons).

Ligne 47 : `min = valeur;` — `min` pointe maintenant vers un autre element du vecteur. C'est valide car `min` et `valeur` ont la meme duree de vie (liee a `v`).

Ligne 71 : `indices.push(index);` — Ajout de l'index au vecteur des resultats.

Lignes 96-100 : `est_trie` compare chaque element avec le suivant. La boucle va de `0` a `v.len() - 2` (car on accede aussi a `i + 1`).

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.enumerate()</code>	Ajoute l' index	Aucun	<code>Enumerate<Iter></code>	<code>(0, &a), (1, &b)...</code>
<code>.is_empty()</code>	Teste si vide	Aucun	<code>bool</code>	<code>vec![] .is_empty() -> true</code>
<code>Some(valeur)</code>	Variante Some de Option	Valeur	<code>Option<T></code>	<code>Some(42)</code>
<code>None</code>	Variante None de Option	Aucun	<code>Option<T></code>	Represente l' absence
<code>match</code>	Pattern match-ing	Expression + arms	Type uniforme	Voir ci-dessous
<code>return</code>	Retour anticipatif	Expression	Sort de la fonction	<code>return Some(i);</code>

Pieges frequents

Erreur	Message du compilateur	Explication
<code>v[0]</code> sur vecteur vide	<code>index out of bounds</code>	Panique a l' execution. Verifiez <code>.is_empty()</code> d' abord.
<code>let min = v[0]</code>	Type <code>i32</code> , pas <code>&i32</code>	Si vous voulez une reference, utilisez <code>&v[0]</code> .
Oublier <code>&</code> dans <code>for</code> <code>&valeur</code>	<code>&i32 == i32 mismatch</code>	Le pattern <code>&valeur</code> dereference automatiquement.
<code>for i in 0..v.len()</code> dans <code>est_trie</code>	<code>index out of bounds pour v[i+1]</code>	Utilisez <code>0..(v.len() - 1)</code> .

Defi bonus

Implementez la recherche dichotomique (binary search) sur un vecteur suppose trie. La fonction doit retourner `Option<usize>` et avoir une complexite $O(\log n)$.

Exercice 10 : Decoupage et Analyse de Phrases

Objectifs

- Manipuler les slices de chaines (&str)
- Utiliser `.split()`, `.trim()`, `.contains()`
- Comprendre la difference entre `String` et `&str`
- Pratiquer le traitement de texte avance

Enonce

Creez un programme d'analyse de phrases qui decoupe, nettoie, et analyse des textes.

Questions

1. Ecrivez `compter_phrases` qui compte le nombre de phrases (separees par `.`, `!`, `?`).
2. Ecrivez `extraire_mots` qui retourne un `Vec<&str>` de tous les mots.
3. Ecrivez `mot_le_plus_long` qui trouve le mot le plus long.
4. Ecrivez `inverser_mots` qui inverse l'ordre des mots dans la phrase.
5. Ecrivez `mettre_en_titre` qui met la premiere lettre de chaque mot en majuscule.
6. Testez avec plusieurs phrases.

Indices

- `.split(|c| c == '.' || c == '!' || c == '?')` separe par plusieurs delimites.
- `.split_whitespace()` separe par tout espace.
- `.chars().next()` donne le premier caractere.
- `.to_uppercase()` convertit en majuscules.

Correction complete

```
fn main() {
    let texte: &str = "Le langage Rust est fascinant. Il est aussi tres performant! \
        Vous l'appreriez vraiment?";

    println!("Texte original :\n{}\n", texte);

    // 1. Compter les phrases
    let nb_phrases = compter_phrases(texte);
    println!("Nombre de phrases : {}", nb_phrases);

    // 2. Extraire les mots
    let mots = extraire_mots(texte);
    println!("Mots trouves : {:?}", mots);
    println!("Nombre de mots : {}", mots.len());

    // 3. Mot le plus long
    match mot_le_plus_long(&mots) {
        Some(mot) => println!("Mot le plus long : \"{}\" ({} lettres)", mot, mot.len()),
        None => println!("Aucun mot trouve"),
    }
}
```

```

    }

    // 4. Inverser l'ordre des mots
    let inverse = inverser_mots(&mots);
    println!("Mots inverses : {}", inverse);

    // 5. Mettre en titre
    let titre = mettre_en_titre(&mots);
    println!("En titre : {}", titre);
}

/// Compte le nombre de phrases dans un texte
/// Les phrases sont separees par '.', '!' ou '?'
fn compter_phrases(texte: &str) -> usize {
    let mut compteur = 0;

    for phrase in texte.split(|c: char| c == '.' || c == '!' || c == '?') {
        let nettoye = phrase.trim();
        if !nettoye.is_empty() {
            compteur += 1;
        }
    }

    compteur
}

/// Extrait tous les mots d'un texte
/// Retourne un Vec de &str (slices referencees sur le texte original)
fn extraire_mots(texte: &str) -> Vec<&str> {
    texte.split_whitespace().collect()
}

/// Trouve le mot le plus long dans un vecteur de mots
/// Parametre :
/// - mots : reference sur Vec<&str>
/// Retourne :
/// - Option<&str> : le mot le plus long, ou None
fn mot_le_plus_long(mots: &Vec<&str>) -> Option<&str> {
    let mut plus_long: Option<&str> = None;

    for &mot in mots.iter() {
        match plus_long {
            None => plus_long = Some(mot),
            Some(actuel) => {
                if mot.len() > actuel.len() {
                    plus_long = Some(mot);
                }
            }
        }
    }

    plus_long
}

```

```

    plus_long
}

/// Inverse l'ordre des mots
/// Parametre :
/// - mots : reference sur Vec<&str>
/// Retourne :
/// - String avec les mots dans l'ordre inverse
fn inverser_mots(mots: &Vec<&str>) -> String {
    let mut resultat = String::new();

    // Iterer en sens inverse
    for (i, &mot) in mots.iter().rev().enumerate() {
        resultat.push_str(mot);
        if i < mots.len() - 1 {
            resultat.push(' ');
        }
    }

    resultat
}

/// Met la premiere lettre de chaque mot en majuscule
/// Parametre :
/// - mots : reference sur Vec<&str>
/// Retourne :
/// - String formatee en titre
fn mettre_en_titre(mots: &Vec<&str>) -> String {
    let mut resultat = String::new();

    for (i, &mot) in mots.iter().enumerate() {
        if i > 0 {
            resultat.push(' ');
        }

        let mut chars = mot.chars();
        match chars.next() {
            Some(premier) => {
                // Premiere lettre en majuscule
                for c in premier.to_uppercase() {
                    resultat.push(c);
                }
                // Reste du mot tel quel
                for c in chars {
                    resultat.push(c);
                }
            }
            None => {} // Mot vide
        }
    }
}

```

```
    resultat
}
```

Explication detaillee ligne par ligne

Ligne 28 : `texte.split(|c: char| c == '.' || c == '!' || c == '?')` — `.split()` accepte un closure qui determine si un caractere est un delimitateur. Le parametre `c` est de type `char`.

Ligne 30 : `let nettoquee = phrase.trim();` — `.trim()` supprime les espaces au debut et a la fin de chaque phrase. Sans cela, les espaces entre les phrases seraient comptes comme des phrases vides.

Ligne 40 : `texte.split_whitespace().collect()` — `.split_whitespace()` gere tous les types d'espaces (espaces, tabulations, retours a la ligne). Le resultat est un iterateur de `&str` (slices du texte original).

Ligne 50-51 : `let mut plus_long: Option<&str> = None;` — L'`Option` stocke une **reference** sur un `&str`. C'est valide car ces references ont la meme duree de vie que le vecteur `mots`.

Lignes 53-61 : `match plus_long { ... }` — Pattern matching sur l'`Option`. Si `None` (premier mot), on initialise. Si `Some(actuel)`, on compare les longueurs.

Lignes 74-80 : `inverser_mots` utilise `.rev()` pour inverser l'ordre. Le pattern `(i, &mot)` avec `.enumerate()` donne l'index et le mot dereference. L'index sert a ne pas ajouter d'espace apres le dernier mot.

Lignes 97-106 : `mettre_en_titre` traite le premier caractere separement. `.to_uppercase()` retourne un iterateur car certaines majuscules Unicode sont multi-caracteres. On itere donc dessus avec `for c in premier.to_uppercase()`.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.split(closure)</code>	Separe selon une condition	Closure <code>FnMut(char) -> bool</code>	Split (iterateur)	<code>"a,b".split(c c == ',')</code>
<code>.split_whitespace()</code>	Separe par espaces	Aucun	SplitWhitespace	<code>"a b\\t c".split_whitespace()</code>
<code>.trim()</code>	Supprime espaces debut/fin	Aucun	<code>&str</code>	<code>" hello ".trim() -> "hello"</code>
<code>.is_empty()</code>	Teste si vide	Aucun	<code>bool</code>	<code>"".is_empty() -> true</code>
<code>.to_uppercase()</code>	Convertit en majuscules	Aucun	ToUppercase (iterateur)	<code>'a'.to_uppercase() -> 'A'</code>
<code>.rev()</code>	Inverse l'iterateur	Aucun	Rev<Iter>	<code>[1,2,3].iter().rev()</code>

Table 21 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.enumerate()</code>	Ajoute l'index	Aucun	<code>Enumerate</code>	<code>(0, &a), (1, &b)...</code>
<code>match</code>	Pattern matching	Expression	Type uniforme	Voir code

Pieges frequents

Erreur	Message du compilateur	Explication
<code>mots: Vec<&str></code> avec <code>texte</code> local	<code>dangling reference</code>	Les <code>&str</code> doivent référencer des données qui vivent assez longtemps.
Oublier <code>.trim()</code> dans <code>compter_phrases</code>	Compte les phrases vides	Les delimitateurs consecutifs produisent des chaines vides.
<code>mot.len()</code>	Compte les octets, pas les caracteres	Pour l' Unicode, utilisez <code>.chars().count()</code> .
<code>Some(mot)</code> ou <code>mot</code> est <code>&str</code>	Lifetime issues	Assurez-vous que les references ont des durees de vie compatibles.

Defi bonus

Ajoutez une fonction `generer_acrostiche` qui prend un `Vec<&str>` et retourne une `String` composee de la premiere lettre de chaque mot.

Exercices 11 a 15 : Ownership et Borrowing

Exercice 11 : Echange de Valeurs et Ownership

Objectifs

- Comprendre le transfert de propriete (move semantics)
- Utiliser le shadowing et la redeclaration
- Distinguer copy types et move types
- Maitriser l'echange de valeurs entre variables

Enonce

Etudiez les mecanismes d'ownership en creant un programme qui echange et manipule des valeurs de differents types.

Questions

1. Declarez une `String` `s1` et assignez-la a `s2`. Que se passe-t-il ? Affichez `s2`.
2. Essayez d'afficher `s1` apres l'assignation. Observez l'erreur.
3. Creez une fonction `prendre_propriete` qui prend une `String` et ne retourne rien.
4. Passez `s2` a cette fonction. Essayez d'utiliser `s2` apres l'appel.
5. Creez une fonction `creer_string` qui retourne une `String`.
6. Declarez un `i32` et montrez qu'il peut etre copie sans probleme (type `Copy`).
7. Utilisez `.clone()` pour copier explicitement une `String`.
8. Ecrivez une fonction `echanger` qui echange deux `String` en utilisant le return pattern.

Indices

- Les types qui implementent `Copy` sont copies automatiquement.
- `String` ne implemente pas `Copy` — elle est “movee”.
- `.clone()` cree une copie profonde (nouvelle allocation memoire).
- Une fonction peut retourner plusieurs valeurs avec un tuple.

Correction complete

```
fn main() {
    // =====
    // 1. Assignation et transfert de propriete (MOVE)
    // =====
    let s1 = String::from("Hello");
    println!("s1 = \"{}\"", s1);

    let s2 = s1; // TRANSFERT DE PROPRIETE : s1 n'est plus valide
    println!("s2 = \"{}\" (apres transfert depuis s1)", s2);

    // 2. Si on decommente la ligne suivante, le compilateur refuse :
    // println!("s1 = {}", s1);
    // ERREUR : "borrow of moved value: s1"

    // =====
    // 3. Fonction qui prend la propriete
    // =====
    println!("\n--- Passage a une fonction ---");
    prendre_propriete(s2);
    // s2 n'est plus valide ici !
    // println!("s2 = {}", s2); // ERREUR

    // =====
    // 5. Fonction qui retourne une String (transfert de retour)
    // =====
    println!("\n--- Retour de fonction ---");
    let s3 = creer_string();
    println!("s3 = \"{}\" (creee et retournee par fonction)", s3);

    // =====
    // 6. Types Copy : les entiers sont copies, pas deplaces
```

```

// =====
println!("\n--- Types Copy (i32) ---");
let a = 42;
let b = a; // COPIE implicite car i32 implemente Copy
println!("a = {}, b = {} (les deux sont valides)", a, b);

let c = a; // On peut copier a autant de fois qu'on veut
println!("a = {}, c = {} (a est toujours valide)", a, c);

// =====
// 7. Clonage explicite d'une String
// =====
println!("\n--- Clonage explicite ---");
let original = String::from("Original");
let copie = original.clone(); // Copie profonde
println!("original = \"{}\"", original);
println!("copie = \"{}\" (clone explicite)", copie);

// original et copie sont TOUS LES DEUX valides car on a clone

// =====
// 8. Echange de deux String via retour de tuple
// =====
println!("\n--- Echange de valeurs ---");
let mut x = String::from("Premier");
let mut y = String::from("Second");
println!("Avant echange : x = \"{}\", y = \"{}\"", x, y);

let (x, y) = echanger(x, y);
println!("Apres echange : x = \"{}\", y = \"{}\"", x, y);
}

/// 3. Fonction qui prend la propriete d'une String
/// Parametre :
/// - s : String (ownership transferee a la fonction)
/// Apres cet appel, l'appelant ne peut plus utiliser s
fn prendre_propriete(s: String) {
    println!(" Fonction a reçu : \"{}\"", s);
    // s est detruite (droppee) a la fin de cette fonction
}

/// 5. Fonction qui cree et retourne une String
/// La propriete de la String retournee est transferee a l'appelant
fn creer_string() -> String {
    let nouvelle = String::from("Creee dans la fonction");
    println!(" Fonction cree : \"{}\"", nouvelle);
    nouvelle // Transfert de propriete a l'appelant
}

/// 8. Echange deux String en utilisant un tuple de retour
/// Parametres :

```

```
/// - a : String (premiere valeur)
/// - b : String (deuxieme valeur)
/// Retourne :
/// - (String, String) : les valeurs echangees
fn echanger(a: String, b: String) -> (String, String) {
    println!(" Fonction echange recoit : \"{}\" et \"{}\"", a, b);
    (b, a) // Retourne dans l'ordre inverse = echange
}
```

Explication detaillee ligne par ligne

Ligne 7 : `let s1 = String::from("Hello");` — Creation d’une String sur le tas (heap). `s1` est le **proprietaire** de cette allocation memoire.

Ligne 9 : `let s2 = s1;` — **Transfert de propriete** (move). La propriete de la String est transferee de `s1` a `s2`. La variable `s1` n’est plus valide apres cette ligne. Rust “invalide” `s1` pour eviter le **double free** (liberation double de memoire).

Ligne 15 : `prendre_propriete(s2);` — La propriete de `s2` est transferee a la fonction. La fonction devient proprietaire et la String sera detruite a la fin de la fonction (appel implicite de `drop`).

Ligne 29 : `let s3 = creer_string();` — La fonction `creer_string` cree une String et la retourne. La propriete est transferee de la fonction vers `s3` dans le `main`.

Ligne 36 : `let b = a;` — Pour les types Copy comme `i32`, l’assignation **copie** la valeur. Les deux variables restent valides. Rust copie les 4 octets de `a` dans `b`.

Ligne 49 : `let copie = original.clone();` — `.clone()` effectue une **copie profonde** de la String. Une nouvelle allocation memoire est faite sur le tas, et le contenu est copie. Les deux String sont independantes.

Lignes 69-73 : `echanger` retourne un tuple `(b, a)`. Les deux String sont transferees ensemble. L’appelant peut les destructure avec `let (x, y) = echanger(x, y);`.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>String::from(s)</code>	Cree une String depuis <code>&str</code>	<code>&str</code>	<code>String</code>	<code>String::from("hello")</code>
<code>.clone()</code>	Copie profonde	Aucun	<code>Self</code>	<code>s1.clone()</code>
<code>(a, b)</code>	Tuple	Elements	<code>(T, U)</code>	<code>(1, "hello")</code>

Pieges frequents

Erreur	Message du compilateur	Explication
Utiliser <code>s1</code> apres <code>let s2 = s1;</code>	<code>borrow of moved value: s1</code>	La propriete a ete transferee. <code>s1</code> n’ est plus valide.

Table 24 – continued

Erreur	Message du compilateur	Explication
Passer une <code>String</code> a une fonction puis la reutiliser	<code>value moved here</code>	La fonction a pris possession. Utilisez <code>&str</code> pour emprunter.
Oublier le <code>mut</code> dans <code>let (x, y) = echanger(x, y)</code>	<code>cannot assign twice to immutable variable</code>	Le shadowing recree une variable, mais il faut <code>mut</code> si on modifie.

Defi bonus

Implementez une fonction `rotation_gauche` qui prend un `Vec<String>` et retourne un nouveau `Vec<String>` ou chaque element est decale d'une position vers la gauche (le premier va a la fin). La fonction doit transferer proprement l'ownership de chaque `String`.

Exercice 12 : References Immuables et Partage

Objectifs

- Maitriser les references immuables (`&T`)
- Comprendre le partage de references immuables
- Distinguer emprunt et propriete
- Utiliser les references comme parametres de fonction

Enonce

Creez un programme qui utilise des references immuables pour lire des donnees sans en prendre possession.

Questions

1. Creez une `String` et passez-la par reference a une fonction `afficher_longueur`.
2. Verifiez que la `String` est toujours utilisable apres l'appel.
3. Creez plusieurs references immuables simultanees sur la meme `String`.
4. Ecrivez une fonction `comparer_longueurs` qui prend deux `&str` et les compare.
5. Ecrivez une fonction `trouver_plus_long` qui retourne une reference sur la chaine la plus longue.
6. Montrez que plusieurs references immuables peuvent coexister.

Indices

- `&String` se convertit automatiquement en `&str` (deref coercion).
- On peut avoir autant de `&T` que l'on veut simultanement.
- Les references doivent etre valides (la donnee referencee doit vivre plus longtemps).

Correction complete

```
fn main() {
    // =====
    // 1. Passage par reference immutable
    // =====
```

```

let message = String::from("Bonjour le monde");
println!("Message : \"{}\"", message);

// Passage par reference : la fonction EMPRUNTE la String
afficher_longueur(&message);

// 2. La String est toujours utilisable car on a juste emprunte
println!("Message encore utilisable : \"{}\"", message);

// =====
// 3. References immuables multiples simultanees
// =====
println!("\n--- References multiples ---");
let ref1 = &message;
let ref2 = &message;
let ref3 = &message;

// Les trois references coexistent
println!("ref1 : \"{}\"", ref1);
println!("ref2 : \"{}\"", ref2);
println!("ref3 : \"{}\"", ref3);

// =====
// 4. Comparaison de deux &str
// =====
println!("\n--- Comparaison ---");
let chaine1 = String::from("Pomme");
let chaine2 = String::from("Banane");

comparer_longueurs(&chaine1, &chaine2);

// =====
// 5. Retour d'une reference
// =====
println!("\n--- Reference retournee ---");
let s1 = String::from("court");
let s2 = String::from("tres tres tres long");

let plus_longue = trouver_plus_long(&s1, &s2);
println!("La plus longue chaine est : \"{}\"", plus_longue);

// =====
// 6. Utilisation dans une portee limitee
// =====
println!("\n--- Portee des references ---");
let texte = String::from("Rust est genial");

{
    let emprunt = &texte; // Reference creee dans la portee interne
    println!("Dans la portee interne : \"{}\"", emprunt);
} // L'emprunt se termine ici

```

```

    // On peut re-emprunter car l'emprunt precedent est termine
    let nouvel_emprunt = &texte;
    println!("Apres la portee : \"{}\"", nouvel_emprunt);

    // Et meme plusieurs a la fois
    let emprunt_a = &texte;
    let emprunt_b = &texte;
    println!("Deux emprunts simultanes : \"{}\" et \"{}\"", emprunt_a, emprunt_b);
}

/// Affiche la longueur d'une chaine empruntee
/// Parametre :
/// - s : &String (reference immutable)
/// La fonction emprunte la String, ne prend pas possession
fn afficher_longueur(s: &String) {
    println!(" Longueur de \"{}\" : {} caracteres", s, s.len());
    // s est une reference : la String originale n'est pas detruite ici
}

/// Compare les longueurs de deux chaines
/// Parametres :
/// - a : &str (emprunte la premiere chaine)
/// - b : &str (emprunte la deuxieme chaine)
fn comparer_longueurs(a: &str, b: &str) {
    println!(" \"{}\" fait {} caracteres", a, a.len());
    println!(" \"{}\" fait {} caracteres", b, b.len());

    if a.len() > b.len() {
        println!(" \"{}\" est plus longue", a);
    } else if b.len() > a.len() {
        println!(" \"{}\" est plus longue", b);
    } else {
        println!(" Les deux chaines ont la meme longueur");
    }
}

/// Retourne une reference sur la chaine la plus longue
/// Parametres :
/// - a : &str
/// - b : &str
/// Retourne :
/// - &str : reference sur la chaine la plus longue
///
/// IMPORTANT : la reference retournee a la meme duree de vie
/// que les parametres (implicite ici)
fn trouver_plus_long<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() {
        a // Retourne la reference sur a
    } else {
        b // Retourne la reference sur b
    }
}

```

```
}  
}
```

Explication detaillee ligne par ligne

Ligne 9 : `afficher_longueur(&message);` — Le `&` cree une **reference immuable**. La fonction emprunte `message` sans en prendre possession. La `String` reste propriete du `main`.

Lignes 18-21 : `let ref1 = &message;` — Trois references immuables sont creees simultanement sur la meme donnee. C'est **autorise** en Rust car les references immuables ne peuvent pas modifier la donnee. Il n'y a aucun risque de conflit.

Ligne 33 : `comparer_longueurs(&chaine1, &chaine2);` — Deref coercion : `&String` est automatiquement converti en `&str`. La fonction recoit des `&str`.

Lignes 42-44 : `trouver_plus_long(&s1, &s2)` — Cette fonction retourne une **reference**. La duree de vie de la reference retournee est liee a celle des parametres (notation `'a` explicite ici pour la pedagogie).

Lignes 55-58 : Portee interne. La reference `emprunt` est creee dans un bloc `{}`. Elle est automatiquement detruite a la fin du bloc. Apres cela, on peut creer de nouvelles references sans conflit.

Ligne 66 : `fn trouver_plus_long<'a>(a: &'a str, b: &'a str) -> &'a str {` — Notation explicite des **durees de vie** (lifetimes). Le `'a` indique que la reference retournee vit aussi longtemps que la plus courte des deux references d'entree.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>&variable</code>	Cree une reference	Variable	<code>&T</code>	<code>let r = &s;</code>
<code>*reference</code>	Dereference (acces a la valeur)	<code>Reference</code>	<code>T</code>	<code>*r</code> pour acceder a la valeur
<code>'a</code>	Notation de duree de vie	-	-	<code>&'a str</code>

Pieges frequents

Erreur	Message du compilateur	Explication
<code>&mut</code> alors qu' on veut juste lire	<code>cannot borrow as mutable</code>	Utilisez <code>&</code> pour un emprunt immuable.
Reference sur une variable locale	<code>does not live long enough</code>	La reference doit vivre moins longtemps que la donnee.
Retourner une reference sur une variable locale	<code>returns a reference to data owned by the current function</code>	Ne retournez jamais une reference sur une variable locale de la fonction.

Defi bonus

Ecrivez une fonction `trouver_plus_long_dans_slice` qui prend un `&[&str]` (slice de references sur des chaines) et retourne `Option<&str>` — la chaine la plus longue dans la slice.

Exercice 13 : Slices de Base

Objectifs

- Comprendre les slices (`&[T]` et `&str`)
- Créer des slices a partir de vecteurs et de String
- Manipuler les slices avec des index de debut et de fin
- Passer des slices comme parametres de fonction

Enonce

Explorez les slices en creant un programme qui decoupe et analyse des portions de donnees.

Questions

1. Creez un `Vec<i32>` de 10 elements et extrayez-en une slice des elements 2 a 5.
2. Ecrivez une fonction `afficher_slice` qui prend `&[i32]` et affiche ses elements.
3. Creez une `String` et extrayez une slice `&str` des caracteres 0 a 5.
4. Ecrivez une fonction `somme_slice` qui calcule la somme d'une slice.
5. Ecrivez une fonction `premier_element` qui retourne `Option<&i32>`.
6. Montrez qu'une slice "emprunte" le vecteur original.

Indices

- `&vecteur[2..6]` cree une slice des elements d'index 2 a 5 (le 6 est exclusif).
- `&chaine[0..5]` cree une slice de chaine (`&str`).
- Les slices sont toujours des references — elles n'ont pas de propriete.
- Une fonction qui prend `&[T]` peut recevoir un vecteur, un tableau, ou une slice.

Correction complete

```
fn main() {  
    // =====  
    // 1. Slice d'un Vec<i32>  
    // =====  
    println!("=== Slices de vecteurs ===");  
    let nombres: Vec<i32> = vec![10, 20, 30, 40, 50, 60, 70, 80, 90, 100];  
    println!("Vecteur complet : {:?}", nombres);  
  
    // Extraction d'une slice : elements d'index 2 a 5 (20, 30, 40, 50)  
    let portion: &[i32] = &nombres[2..6];  
    println!("Slice [2..6] : {:?}", portion);  
  
    // Autres exemples de slices  
    let debut: &[i32] = &nombres[..4];           // Du debut a l'index 3  
    let fin: &[i32] = &nombres[7..];             // De l'index 7 jusqu'a la fin
```



```

let complet: &[i32] = &nombres[..]; // Tout le vecteur

println!("Slice[..4] : {:?}", debut);
println!("Slice[7..] : {:?}", fin);
println!("Slice[..] : {:?}", complet);

// 2. Fonction qui prend une slice
println!("\n--- Fonction afficher_slice ---");
afficher_slice(portion);
afficher_slice(&nombres);
afficher_slice(debut);

// =====
// 3. Slice de chaine (&str)
// =====
println!("\n=== Slices de chaines ===");
let phrase = String::from("Bonjour le monde Rust");
println!("Phrase complete : {:?}", phrase);

// Extraction d'une sous-chaine
let mot1: &str = &phrase[0..7]; // "Bonjour"
let mot2: &str = &phrase[8..10]; // "le"
let mot3: &str = &phrase[11..16]; // "monde"

println!("Mots extraits : {:?}", "\{:}\", \{:}\", \{:}\\"", mot1, mot2, mot3);

// =====
// 4. Fonction qui travaille sur une slice
// =====
println!("\n--- Fonction somme_slice ---");
let total_complet = somme_slice(&nombres);
let total_portion = somme_slice(portion);
println!("Somme complete : {:?}", total_complet);
println!("Somme de la portion [2..6] : {:?}", total_portion);

// =====
// 5. Premier element d'une slice
// =====
println!("\n--- Fonction premier_element ---");
match premier_element(portion) {
    Some(&val) => println!("Premier element de la slice : {:?}", val),
    None => println!("La slice est vide"),
}

let vide: &[i32] = &[];
match premier_element(vide) {
    Some(&val) => println!("Premier element : {:?}", val),
    None => println!("Slice vide : pas de premier element"),
}

// =====

```

```

// 6. Demonstration : la slice emprunte le vecteur
// =====
println!("\n== Emprunt via slice ==");
let donnees = vec![1, 2, 3, 4, 5];

// La slice emprunte 'donnees'
let emprunt: &[i32] = &donnees[1..4];
println!("Slice empruntée : {:?}", emprunt);

// 'donnees' est toujours utilisable (emprunt immuable)
println!("Vecteur original toujours valide : {:?}", donnees);
println!("La slice voit toujours : {:?}", emprunt);
}

/// Affiche les elements d'une slice
/// Parametre :
///   - slice : &[i32] (slice de i32)
///
/// Cette fonction accepte n'importe quelle slice de i32,
/// qu'elle vienne d'un Vec, d'un tableau, ou d'une autre slice.
fn afficher_slice(slice: &[i32]) {
    println!(" Slice recue ({} elements) : {:?}", slice.len(), slice);
}

/// Calcule la somme des elements d'une slice
/// Parametre :
///   - slice : &[i32]
/// Retourne :
///   - i32 : la somme de tous les elements
fn somme_slice(slice: &[i32]) -> i32 {
    let mut total = 0;
    for &valeur in slice.iter() {
        total += valeur;
    }
    total
}

/// Retourne le premier element d'une slice
/// Parametre :
///   - slice : &[i32]
/// Retourne :
///   - Option<i32> : Some(&premier) ou None si vide
fn premier_element(slice: &[i32]) -> Option<i32> {
    if slice.is_empty() {
        None
    } else {
        Some(&slice[0])
    }
}

```

Explication detaillee ligne par ligne

Ligne 10 : `let portion: &[i32] = &nombres[2..6];` — Creation d’une slice. La syntaxe `[2..6]` signifie “de l’index 2 inclus jusqu’a l’index 6 exclus”. La slice contient les elements d’index 2, 3, 4, 5. Le `&` est obligatoire car une slice est toujours une reference.

Lignes 13-15 : Trois formes de slices :

- `..4` : du debut jusqu’a l’index 3 (4 exclus)
- `7..` : de l’index 7 jusqu’a la fin
- `..` : du debut jusqu’a la fin (tout)

Ligne 37 : `let mot1: &str = &phrase[0..7];` — Slice de chaine. Le type est `&str`. Attention : cette indexation est en **octets**, pas en caracteres Unicode. “Bonjour” fait 7 octets en ASCII.

Ligne 64 : `fn afficher_slice(slice: &[i32]) {` — Une fonction qui prend une slice peut recevoir n’importe quelle donnee sequentielle : un vecteur (`&nombres`), un tableau, ou une slice existante.

Ligne 80 : `for &valeur in slice.iter() {` — Iteration sur la slice. Le pattern `&valeur` dereference automatiquement.

Ligne 96 : `Some(&slice[0])` — Retourne une **reference** sur le premier element de la slice.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>&v[a..b]</code>	Extraction de slice	Indices debut et fin (exclusif)	<code>&[T]</code>	<code>&v[1..3]</code>
<code>&v[..n]</code>	Slice depuis le debut	Fin (exclusif)	<code>&[T]</code>	<code>&v[..3]</code>
<code>&v[n..]</code>	Slice jusqu’ a la fin	Debut	<code>&[T]</code>	<code>&v[3..]</code>
<code>&v[..]</code>	Slice complete	Aucun	<code>&[T]</code>	<code>&v[..]</code>
<code>.is_empty()</code>	Teste si la slice est vide	Aucun	<code>bool</code>	<code>[].is_empty()</code>

Pieges frequents

Erreur	Message du compilateur	Explication
<code>&nombres[2..6]</code> hors bornes	<code>range index out of bounds</code>	Verifiez que les indices sont valides.
Slice sur String non-ASCII	<code>byte index is not a char boundary</code>	Les slices de <code>&str</code> utilisent des index d’ octets, pas de caracteres.

Table 28 – continued

Erreur	Message du compilateur	Explication
&nombres et modification simultanee	cannot borrow as mutable	Une slice emprunte immuablement. Pas de mutation simultanee.

Defi bonus

Ecrivez une fonction `tranche_centrale` qui prend une slice et retourne une sous-slice contenant les elements du milieu (sans le premier ni le dernier). Gerez le cas ou la slice a moins de 2 elements.

Exercice 14 : Mutable References

Objectifs

- Comprendre les references mutables (`&mut T`)
- Respecter la regle : une seule reference mutable a la fois
- Modifier des donnees via une reference
- Combiner emprunts mutables et immuables

Enonce

Creez un programme qui utilise des references mutables pour modifier des donnees.

Questions

1. Creez une `String` mutable et modifiez-la via une reference mutable.
2. Ecrivez une fonction `ajouter_suffixe` qui prend `&mut String` et ajoute du texte.
3. Ecrivez une fonction `doubler_valeurs` qui prend `&mut Vec<i32>` et double chaque element.
4. Montrez la regle : on ne peut pas avoir deux `&mut` simultanement.
5. Montrez la regle : on ne peut pas avoir `&mut` et `&` simultanement.
6. Ecrivez une fonction `echanger_premier_dernier` qui echange les deux elements extremes.

Indices

- `&mut variable` cree une reference mutable.
- La regle fondamentale : OU BIEN une reference mutable, OU BIEN plusieurs references immuables.
- Les references mutables sont **exclusives**.
- Les scopes peuvent aider a gerer les durees de vie des references.

Correction complete

```
fn main() {
    // =====
    // 1. Reference mutable sur une String
    // =====
    println!("=== References mutables ===");
    let mut message = String::from("Hello");
    println!("Avant modification : \"{}\"", message);
```

```

{
    let ref_mut: &mut String = &mut message;
    ref_mut.push_str(" World");
    println!("Dans le bloc : \"{}\"", ref_mut);
} // La reference mutable se termine ici

println!("Apres modification : \"{}\"", message);

// =====
// 2. Fonction avec reference mutable
// =====
println!("--- Fonction ajouter_suffixe ---");
let mut texte = String::from("Rust");
println!("Avant : \"{}\"", texte);
ajouter_suffixe(&mut texte, " est genial");
println!("Apres : \"{}\"", texte);

// =====
// 3. Fonction qui modifie un Vec
// =====
println!("--- Fonction doubler_valeurs ---");
let mut nombres = vec![1, 2, 3, 4, 5];
println!("Avant : {:?}", nombres);
doubler_valeurs(&mut nombres);
println!("Apres : {:?}", nombres);

// =====
// 4. Regle : une seule reference mutable a la fois
// =====
println!("--- Regle : une seule &mut ---");
let mut valeur = 42;

{
    let r1 = &mut valeur;
    *r1 += 10; // Dereferencement pour modifier la valeur
    println!("r1 a modifie : {}", r1);
} // r1 se termine ici

{
    let r2 = &mut valeur;
    *r2 *= 2;
    println!("r2 a modifie : {}", r2);
} // r2 se termine ici

println!("Valeur finale : {}\n", valeur);

// Le code suivant serait INTERDIT :
// let r1 = &mut valeur;
// let r2 = &mut valeur; // ERREUR : cannot borrow as mutable more than once

```

```

// =====
// 5. Regle : pas de &mut et & simultanes
// =====
println!("--- Regle : pas de &mut et & simultanes ---");
let mut data = String::from("donnees");

{
    let ref_imm = &data;
    println!("Reference immuable : \"{}\"", ref_imm);
} // L'emprunt immuable se termine

// Maintenant on peut creer une reference mutable
let ref_mut = &mut data;
ref_mut.push('!');
println!("Reference mutable apres : \"{}\"", ref_mut);

// =====
// 6. Echanger premier et dernier elements
// =====
println!("--- Fonction echanger_premier_dernier ---");
let mut elements = vec![10, 20, 30, 40, 50];
println!("Avant echange : {:?}", elements);
echanger_premier_dernier(&mut elements);
println!("Apres echange : {:?}", elements);
}

/// Ajoute un suffixe a une String via une reference mutable
/// Parametres :
/// - s : &mut String (reference mutable)
/// - suffixe : &str (reference immuable sur le texte a ajouter)
fn ajouter_suffixe(s: &mut String, suffixe: &str) {
    s.push_str(suffixe);
    // s est une reference mutable : la modification affecte l'original
}

/// Double chaque element d'un vecteur
/// Parametre :
/// - v : &mut Vec<i32> (reference mutable sur le vecteur)
fn doubler_valeurs(v: &mut Vec<i32>) {
    for i in 0..v.len() {
        v[i] *= 2; // Modification via indexation mutable
    }
}

/// Echange le premier et le dernier element d'un vecteur
/// Parametre :
/// - v : &mut Vec<i32>
/// Ne fait rien si le vecteur a moins de 2 elements
fn echanger_premier_dernier(v: &mut Vec<i32>) {
    if v.len() < 2 {
        return; // Rien a faire
    }
}

```

```
}

let dernier_index = v.len() - 1;

// Utilisation de std::mem::swap pour echanger
// On doit acceder aux elements comme des references mutables
let ptr = v.as_mut_ptr();
unsafe {
    std::ptr::swap(ptr, ptr.add(dernier_index));
}

// VERSION SANS UNSAFE (plus idiomatique) :
// v.swap(0, dernier_index);
}
```

Explication detaillee ligne par ligne

Ligne 10 : `let ref_mut: &mut String = &mut message;` — Creation d’une reference mutable. Le `mut` est necessaire dans trois endroits : sur la variable `message`, sur `ref_mut`, et dans le type `&mut`.

Ligne 11 : `ref_mut.push_str(" World");` — Modification de la `String` via la reference. La methode `push_str` est appelee sur la reference mutable.

Lignes 14-18 : Le bloc `{}` limite la duree de vie de `ref_mut`. Une fois le bloc termine, `ref_mut` n’existe plus et on peut reutiliser `message`.

Ligne 38 : `ajouter_suffixe(&mut texte, " est genial");` — La fonction prend une reference mutable (`&mut texte`) et une reference immuable ("`est genial`" est un `&str`).

Lignes 62-76 : Demonstration des regles d’emprunt. Les blocs imbriques garantissent que les references ne se chevauchent pas. On peut avoir des `&mut` successifs, mais pas simultanes.

Lignes 90-95 : On peut passer de `&` a `&mut` uniquement si l’emprunt immuable est termine. Le compilateur verifie que les durees de vie ne se chevauchent pas.

Lignes 113-119 : `echanger_premier_dernier`. La version avec `v.swap(0, dernier_index)` est la plus idiomatique en Rust. La version avec `std::ptr::swap` montre le dereferencement de pointeurs bruts.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>&mut var</code>	Reference mutable	Variable mutable	<code>&mut T</code>	<code>let r = &mut s;</code>
<code>*r</code>	Dereferencement	Reference mutable	Valeur mutable	<code>*r += 1;</code>
<code>.push_str(s)</code>	Ajoute une chaine	<code>&str</code>	<code>()</code>	<code>s.push_str("!");</code>
<code>.swap(i, j)</code>	Echange deux elements	Deux index	<code>()</code>	<code>v.swap(0, 1);</code>

Table 29 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>as_mut_ptr()</code>	Pointeur brut mutable	Aucun	<code>*mut T</code>	Pour operations unsafe

Pieges frequents

Erreur	Message du compilateur	Explication
Deux <code>&mut</code> simultanes	<code>cannot borrow as mutable more than once at a time</code>	Regle fondamentale de Rust.
<code>&mut</code> et <code>&</code> simultanes	<code>cannot borrow as mutable because it is also borrowed as immutable</code>	Emprunt exclusif requis.
Oublier <code>mut</code> sur la variable	<code>cannot borrow as mutable</code>	La variable elle-meme doit etre <code>mut</code> .
Oublier <code>mut</code> sur la reference	<code>cannot borrow as mutable</code>	Le type de reference doit etre <code>&mut T</code> .

Defi bonus

Ecrivez une fonction `trier_deux` qui prend deux `&mut i32` et les echange si necessaire pour que le premier soit `<=` au deuxieme. Cette fonction doit fonctionner meme quand les deux references pointent vers la meme variable (dans ce cas, elle ne fait rien).

Exercice 15 : Combinaison d’Emprunts Mutables et Immuables

Objectifs

- Comprendre les interactions entre `&T` et `&mut T`
- Utiliser les scopes pour gerer les durees de vie
- Comprendre le “borrow checker” de Rust
- Implementer des fonctions qui combinent differents types d’emprunts

Enonce

Creez un programme qui combine emprunts mutables et immuables pour traiter des donnees, en respectant strictement les regles de Rust.

Questions

1. Creez un `Vec<String>` et iterez immuablement dessus.
2. Apres l’iteration, modifiez un element via `&mut`.
3. Ecrivez une fonction `afficher_et_modifier` qui d’abord affiche (emprunt immuable) puis modifie (emprunt mutable) dans des scopes separees.
4. Ecrivez une fonction `trouver_et_remplacer` qui cherche un element et le remplace.
5. Montrez que le borrow checker empeche les acces concurrents invalides.
6. Utilisez des scopes pour “convaincre” le borrow checker.

Indices

- Les emprunts immuables peuvent être multiples mais ne peuvent coexister avec un emprunt mutable.
- Le borrow checker regarde les **durées de vie** (lifetimes), pas seulement les lignes.
- Les scopes {} limitent les durées de vie.
- Après la fin d'un emprunt, un nouvel emprunt d'un type différent est possible.

Correction complete

```
fn main() {  
    // =====  
    // 1. Iteration immuable sur un Vec<String>  
    // =====  
    println!("=== Emprunts combines ===");  
    let mut fruits = vec![  
        String::from("Pomme"),  
        String::from("Banane"),  
        String::from("Cerise"),  
    ];  
    println!("Vecteur initial : {:?}", fruits);  
  
    // Iteration immuable (emprunt immuable temporaire)  
    println!("\n--- Iteration immuable ---");  
    for fruit in &fruits {  
        println!(" Fruit : {:?}", fruit);  
    } // L'emprunt immuable de la boucle se termine ici  
  
    // =====  
    // 2. Modification apres l'iteration  
    // =====  
    println!("\n--- Modification par emprunt mutable ---");  
    fruits[0].push_str(" Rouge"); // Modification du premier element  
    println!("Apres modification : {:?}", fruits);  
  
    // =====  
    // 3. Fonction avec emprunts separes par scope  
    // =====  
    println!("\n--- Fonction afficher_et_modifier ---");  
    let mut mot = String::from("Hello");  
    afficher_et_modifier(&mut mot);  
    println!("Resultat final : {:?}", mot);  
  
    // =====  
    // 4. Fonction trouver et remplacer  
    // =====  
    println!("\n--- Fonction trouver_et_remplacer ---");  
    let mut noms = vec![  
        String::from("Alice"),  
        String::from("Bob"),  
        String::from("Alice"),  
        String::from("Charlie"),  
    ];
```

```

};
println!("Avant : {:?}", noms);
trouver_et_remplacer(&mut noms, "Alice", "Alicia");
println!("Après : {:?}", noms);

// =====
// 5. Demonstration : le borrow checker protege
// =====
println!("\n--- Le borrow checker en action ---");
let mut donnees = vec![10, 20, 30, 40, 50];

// Emprunt immutable dans une portee limitee
{
    let premier = &donnees[0];
    println!("Premier element (emprunte) : {}", premier);
} // L'emprunt se termine

// Maintenant on peut emprunter mutabelement
{
    let premier_mut = &mut donnees[0];
    *premier_mut = 99;
    println!("Premier element modifie : {}", premier_mut);
} // L'emprunt mutable se termine

// On peut maintenant emprunter a nouveau immuablement
println!("Vecteur final : {:?}", donnees);

// =====
// 6. Utilisation avantee des scopes
// =====
println!("\n--- Scopes avances ---");
let mut valeurs = vec![1, 2, 3, 4, 5];

// Scope 1 : lire
{
    let slice = &valeurs[..3];
    println!("Slice lue : {:?}", slice);
}

// Scope 2 : modifier
{
    let slice_mut = &mut valeurs[3..];
    for v in slice_mut.iter_mut() {
        *v *= 10;
    }
    println!("Slice modifiee : {:?}", &valeurs[3..]);
}

// Scope 3 : relire tout
println!("Vecteur complet : {:?}", valeurs);
}

```

```

/// Affiche une String puis la modifie
/// Les emprunts immuable et mutable sont dans des scopes distincts
fn afficher_et_modifier(s: &mut String) {
    // Scope 1 : emprunt immuable pour l'affichage
    {
        let ref_imm = &*s; // Emprunt immuable temporaire
        println!(" Affichage (emprunt immuable) : \"{}\"", ref_imm);
    } // L'emprunt immuable se termine

    // Scope 2 : emprunt mutable pour la modification
    {
        let ref_mut = s; // Emprunt mutable (reborrow)
        ref_mut.push_str(" World!");
        println!(" Modification (emprunt mutable) : \"{}\"", ref_mut);
    }
}

/// Trouve toutes les occurrences d'une valeur et les remplace
/// Parametres :
/// - v : &mut Vec<String> (vecteur a modifier)
/// - ancien : &str (valeur a chercher)
/// - nouveau : &str (valeur de remplacement)
fn trouver_et_remplacer(v: &mut Vec<String>, ancien: &str, nouveau: &str) {
    for element in v.iter_mut() {
        if element == ancien {
            *element = String::from(nouveau);
        }
    }
}

```

Explication detaillee ligne par ligne

Ligne 14-16 : `for fruit in &fruits { ... }` — La boucle `for` emprunte immuablement le vecteur. A chaque iteration, `fruit` est de type `&String`. L'emprunt se termine automatiquement a la fin de la boucle.

Ligne 24 : `fruits[0].push_str(" Rouge");` — Cette ligne modifie le premier element. C'est autorise car l'emprunt immuable de la boucle est termine. Le compilateur sait qu'il n'y a plus d'emprunt actif.

Lignes 67-81 : `afficher_et_modifier` utilise des scopes explicites pour separer l'emprunt immuable de l'emprunt mutable. Sans ces scopes, le compilateur refuserait car `ref_imm` et `ref_mut` ne peuvent pas coexister.

Ligne 85-88 : `trouver_et_remplacer` utilise `.iter_mut()` qui produit des references mutables sur chaque element. Le pattern `*element = ...` dereference et remplace la valeur.

Lignes 101-118 : Demonstration des trois phases : emprunt immuable -> fin -> emprunt mutable -> fin -> reutilisation. Cette sequence respecte les regles de Rust.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.iter_mut()</code>	Iterateur d'emprunt mutable	Aucun	<code>slice::IterMut<T></code>	Produit <code>&mut T</code>
<code>*ref</code>	Dereference	Reference	Valeur	<code>*r = 5;</code>
<code>&*s</code>	Re-emprunt immutable	Reference mutable	Reference immuable	Technique avancee

Pieges frequents

Erreur	Message du compilateur	Explication
Emprunt <code>&</code> puis <code>&mut</code>	<code>cannot borrow as mutable</code>	L' emprunt immuable est encore actif.
<code>ref_imm</code> utilise apres creation de <code>ref_mut</code>	<code>mutable borrow starts here</code>	Les emprunts ne peuvent pas se chevaucher.
<code>for x in &v</code> puis modification	<code>cannot borrow as mutable</code>	L' iterateur emprunte le vecteur.

Defi bonus

Ecrivez une fonction `partition_mut` qui prend `&mut Vec<i32>` et un `seuil: i32`, et reorganise le vecteur pour que tous les elements `< seuil` soient au debut, et les autres a la fin. La fonction doit utiliser uniquement des echanges (pas de nouvelles allocations).

Exercices 16 a 20 : Ownership, Borrowing et Slices avances

Exercice 16 : Slices Avancees — Analyse de Texte

Objectifs

- Maitriser les slices de chaines (`&str`) avec l'Unicode
- Comprendre la difference entre octets et caracteres
- Utiliser `as_bytes()` et `chars()` pour naviguer dans le texte
- Implementer des fonctions qui retournent des slices

Enonce

Creez un programme d'analyse de texte avance qui manipule des slices de chaines avec une attention particuliere a l'Unicode.

Questions

1. Ecrivez `premier_mot` qui retourne le premier mot d'une phrase (slice jusqu'au premier espace).
2. Ecrivez `dernier_mot` qui retourne le dernier mot.
3. Ecrivez `mot_a_position` qui retourne le n-ieme mot.
4. Ecrivez `contient_mot` qui verifie si un mot existe dans le texte.
5. Ecrivez `extraire_phrase` qui retourne une phrase complete (jusqu'a ., !, ?).
6. Comparez le comportement avec du texte ASCII et Unicode.

Indices

- `.find(' ')` retourne `Option<usize>` avec la position du premier espace.
- `.rfind(' ')` cherche depuis la fin.
- `.split_whitespace()` produit des mots.
- Faites attention aux caracteres multi-octets en Unicode.

Correction complete

```
fn main() {
    let phrase = "Le langage Rust est vraiment excellent";
    println!("Phrase : \"{}\\n\"", phrase);

    // 1. Premier mot
    let mot1 = premier_mot(phrase);
    println!("Premier mot : \"{}\\n\"", mot1);

    // 2. Dernier mot
    let mot_dernier = dernier_mot(phrase);
    println!("Dernier mot : \"{}\\n\"", mot_dernier);

    // 3. Mot a une position
    match mot_a_position(phrase, 2) {
        Some(mot) => println!("3eme mot : \"{}\\n\"", mot),
        None => println!("Position invalide"),
    }

    // 4. Contient un mot
    println!("Contient 'Rust' : {}", contient_mot(phrase, "Rust"));
    println!("Contient 'Python' : {}", contient_mot(phrase, "Python"));

    // 5. Extraire la premiere phrase
    let texte = "Premiere phrase. Deuxieme phrase! Troisieme?";
    match extraire_phrase(texte) {
        Some(p) => println!("\nPremiere phrase : \"{}\\n\"", p),
        None => println!("Pas de phrase trouvee"),
    }

    // 6. Test Unicode
    println!("\n=== Test Unicode ===");
    let unicode = "Rust est super et rapide";
    println!("Phrase Unicode : \"{}\\n\"", unicode);
}
```

```

println!("Premier mot Unicode : \"{}\"", premier_mot(unicode));
println!("Dernier mot Unicode : \"{}\"", dernier_mot(unicode));
}

/// Retourne le premier mot d'une phrase (slice jusqu'au premier espace)
/// Parametre :
/// - s : &str
/// Retourne :
/// - &str : slice du premier mot
fn premier_mot(s: &str) -> &str {
    // as_bytes() convertit la chaine en slice d'octets
    let bytes = s.as_bytes();

    // Iteration sur les octets avec leur index
    for (i, &item) in bytes.iter().enumerate() {
        // b' ' est la notation pour l'octet d'espace (32)
        if item == b' ' {
            // Retourne une slice du debut jusqu'a l'espace
            return &s[0..i];
        }
    }

    // Si pas d'espace trouve, toute la chaine est un mot
    s
}

/// Retourne le dernier mot d'une phrase
/// Parametre :
/// - s : &str
/// Retourne :
/// - &str : slice du dernier mot
fn dernier_mot(s: &str) -> &str {
    // as_bytes() pour travailler au niveau des octets
    let bytes = s.as_bytes();

    // Iteration en sens inverse pour trouver le dernier espace
    for (i, &item) in bytes.iter().enumerate().rev() {
        if item == b' ' {
            // Retourne une slice apres le dernier espace
            return &s[(i + 1)..];
        }
    }

    // Si pas d'espace, toute la chaine est le dernier mot
    s
}

/// Retourne le n-ieme mot (0-indexe)
/// Parametres :
/// - s : &str
/// - n : usize (position, 0 = premier mot)

```

```

/// Retourne :
/// - Option<&str> : le mot trouve ou None
fn mot_a_position(s: &str, n: usize) -> Option<&str> {
    // split_whitespace() produit un iterateur de mots
    let mut mots = s.split_whitespace();

    // nth(n) avance l'iterateur de n positions et retourne l'element
    mots.nth(n)
}

/// Verifie si un mot existe dans le texte
/// Parametres :
/// - texte : &str (texte a chercher)
/// - mot : &str (mot a trouver)
/// Retourne :
/// - true si le mot est present, false sinon
fn contient_mot(texte: &str, mot: &str) -> bool {
    // contains() verifie si une sous-chaine est presente
    texte.contains(mot)
}

/// Extrait la premiere phrase (jusqu'a ., ! ou ?)
/// Parametre :
/// - s : &str
/// Retourne :
/// - Option<&str> : la phrase sans la ponctuation finale, ou None
fn extraire_phrase(s: &str) -> Option<&str> {
    // Recherche la premiere ponctuation de fin de phrase
    for (i, c) in s.char_indices() {
        if c == '.' || c == '!' || c == '?' {
            return Some(&s[0..i]);
        }
    }

    // Si aucune ponctuation trouvee, retourne toute la chaine
    if s.is_empty() {
        None
    } else {
        Some(s)
    }
}

```

Explication detaillee ligne par ligne

Ligne 48 : `let bytes = s.as_bytes();` — Convertit la `&str` en `&[u8]` (slice d'octets). Cela permet d'iterer octet par octet, ce qui est plus rapide pour chercher des caracteres ASCII.

Ligne 51 : `for (i, &item) in bytes.iter().enumerate() {` — Iteration sur les octets avec leur index. `item` est un `u8` (dereference par le pattern `&item`).

Ligne 53 : `if item == b' '` { — `b' '` est la notation pour le literal d'octet espace. C'est equivalent a

32u8.

Ligne 55 : `return &s[0..i];` — Retourne une `&str` qui est une slice du debut jusqu'a la position de l'espace. La slice `&str` a la meme duree de vie que `s`.

Ligne 59 : `s` — Si aucun espace n'est trouve, on retourne la chaine entiere. C'est un `&str` qui pointe sur toute la chaine.

Ligne 73 : `.rev()` inverse l'iterateur. On parcourt les octets depuis la fin pour trouver le dernier espace.

Ligne 87 : `.nth(n)` consomme `n` elements de l'iterateur et retourne le `n`-ieme. C'est un raccourci pour `.skip(n).next()`.

Ligne 102 : `.contains(mot)` verifie si `mot` est une sous-chaine de `texte`. Cette methode travaille sur les caracteres Unicode correctement.

Ligne 113 : `.char_indices()` itere sur les caracteres Unicode avec leur position en octets. C'est crucial pour l'Unicode car les caracteres n'occupent pas tous un seul octet.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.as_bytes()</code>	Convertit <code>&str</code> en <code>&[u8]</code>	Aucun	<code>&[u8]</code>	<code>"hello".as_bytes()</code>
<code>.iter().enumerate()</code>	Index + valeur	Aucun	<code>(usize, &T)</code>	<code>(0, &h), (1, &e)...</code>
<code>.rev()</code>	Inverse l' iterateur	Aucun	<code>Rev<Iter></code>	<code>[1,2].iter().rev()</code>
<code>.nth(n)</code>	<code>n</code> -ieme element	<code>usize</code>	<code>Option<T></code>	Consomme l' iterateur
<code>.contains(sub)</code>	Teste sous-chaine	<code>&str</code>	<code>bool</code>	<code>"abc".contains("b")</code>
<code>.char_indices()</code>	Caracteres + positions octets	Aucun	<code>CharIndices</code>	<code>(0, 'h'), (1, 'e')...</code>
<code>.split_whitespace()</code>	Separe par espaces	Aucun	<code>SplitWhitespace</code>	<code>"a b".split_whitespace()</code>

Pieges frequents

Erreur	Message du compilateur	Explication
Slice sur caractere Unicode multi-octets	<code>byte index is not a char boundary</code>	Les slices <code>&str</code> doivent couper aux frontieres de caracteres.

Table 34 – continued

Erreur	Message du compilateur	Explication
<code>.len()</code> pour compter les caracteres	Compte les octets	Utilisez <code>.chars().count()</code> pour les caracteres.
<code>bytes[i] == ' '</code> (char vs u8)	Type mismatch	Utilisez <code>b' '</code> pour les octets.

Defi bonus

Ecrivez une fonction `tous_les_mots` qui retourne un `Vec<&str>` contenant tous les mots, puis implementez `frequence_mots` qui compte combien de fois chaque mot apparait.

Exercice 17 : Mutable References sur des Slices

Objectifs

- Comprendre `&mut [T]`
- Modifier des elements via une slice mutable
- Comprendre la re-borrowing
- Implementer des algorithmes de tri basiques

Enonce

Creez un programme qui utilise des slices mutables pour modifier des portions de donnees.

Questions

1. Ecrivez `inverser_slice` qui inverse l'ordre des elements d'une slice mutable.
2. Ecrivez `remplacer_par` qui remplace tous les elements egaux a une valeur par une autre.
3. Ecrivez `trier_tranche` qui trie une sous-portion d'un vecteur.
4. Ecrivez `multiplier_par_deux` qui double tous les elements.
5. Ecrivez `decaler_gauche` qui decale tous les elements d'une position a gauche.
6. Montrez l'utilisation avec des portions de vecteurs.

Indices

- `&mut v[a..b]` cree une slice mutable.
- `.swap(i, j)` echange deux elements.
- `.sort()` trie une slice mutable.
- La re-borrowing permet d'emprunter une partie deja empruntee mutablement.

Correction complete

```
fn main() {
    // =====
    // 1. Inverser une slice
    // =====
    println!("=== Slices mutables ===");
}
```

```

let mut nombres = vec![1, 2, 3, 4, 5, 6, 7, 8];
println!("Vecteur initial : {:?}", nombres);

inverser_slice(&mut nombres[2..6]);
println!("Après inversion [2..6] : {:?}", nombres);

// Inverser tout le vecteur
inverser_slice(&mut nombres[..]);
println!("Après inversion complète : {:?}", nombres);

// =====
// 2. Remplacer les valeurs
// =====
println!("\n--- Remplacer par ---");
let mut scores = vec![10, 20, 10, 30, 10, 40];
println!("Avant : {:?}", scores);
remplacer_par(&mut scores[..], 10, 99);
println!("Après remplacement 10->99 : {:?}", scores);

// =====
// 3. Trier une tranche
// =====
println!("\n--- Trier tranche ---");
let mut melange = vec![5, 2, 8, 1, 9, 3, 7];
println!("Avant tri : {:?}", melange);
trier_tranche(&mut melange[2..5]);
println!("Après tri [2..5] : {:?}", melange);

// =====
// 4. Multiplier par deux
// =====
println!("\n--- Multiplier par deux ---");
let mut valeurs = vec![1, 2, 3, 4, 5];
println!("Avant : {:?}", valeurs);
multiplier_par_deux(&mut valeurs[..]);
println!("Après *2 : {:?}", valeurs);

// =====
// 5. Decalage a gauche
// =====
println!("\n--- Decaler gauche ---");
let mut lettres = vec!['a', 'b', 'c', 'd', 'e'];
println!("Avant : {:?}", lettres);
decaler_gauche(&mut lettres[..]);
println!("Après decalage : {:?}", lettres);

// =====
// 6. Utilisation avec portions
// =====
println!("\n--- Portions multiples ---");
let mut data = vec![10, 20, 30, 40, 50, 60];

```

```

println!("Initial : {:?}", data);

// On peut scinder le vecteur en deux slices mutables disjointes
let milieu = data.len() / 2;
let (gauche, droite) = data.split_at_mut(milieu);

multiplier_par_deux(gauche);
inverser_slice(droite);

println!("Apres modifications : {:?}", data);
}

/// Inverse l'ordre des elements d'une slice mutable
/// Algorithme : echange les elements symetriques depuis les extremités
/// Parametre :
/// - slice : &mut [i32]
fn inverser_slice(slice: &mut [i32]) {
    let len = slice.len();

    // On parcourt seulement la moitié de la slice
    for i in 0..(len / 2) {
        let j = len - 1 - i; // Position symetrique
        slice.swap(i, j);    // Echange les deux elements
    }
}

/// Remplace toutes les occurrences d'une valeur par une autre
/// Parametres :
/// - slice : &mut [i32] (slice a modifier)
/// - ancienne : i32 (valeur a remplacer)
/// - nouvelle : i32 (nouvelle valeur)
fn remplacer_par(slice: &mut [i32], ancienne: i32, nouvelle: i32) {
    for element in slice.iter_mut() {
        if *element == ancienne {
            *element = nouvelle;
        }
    }
}

/// Trie une sous-tranche d'un vecteur
/// Parametre :
/// - slice : &mut [i32]
fn trier_tranche(slice: &mut [i32]) {
    slice.sort();
}

/// Double tous les elements d'une slice
/// Parametre :
/// - slice : &mut [i32]
fn multiplier_par_deux(slice: &mut [i32]) {
    for element in slice.iter_mut() {

```

```
        *element *= 2;
    }
}

/// Decale tous les elements d'une position a gauche
/// Le dernier element recoit la valeur du premier (avant decalage)
/// Parametre :
/// - slice : &mut [T] (slice generique)
fn decaler_gauche<T: Clone>(slice: &mut [T]) {
    if slice.len() <= 1 {
        return;
    }

    let premier = slice[0].clone();

    for i in 0..(slice.len() - 1) {
        slice[i] = slice[i + 1].clone();
    }

    slice[slice.len() - 1] = premier;
}
```

Explication detaillee ligne par ligne

Ligne 10 : `inverser_slice(&mut nombres[2..6]);` — Creation d’une slice mutable sur une portion du vecteur. Seuls les elements d’index 2 a 5 seront inverses.

Lignes 57-63 : `inverser_slice` utilise l’algorithme classique : on parcourt la moitie de la slice et on echange chaque element avec son symetrique. `.swap(i, j)` est une methode standard des slices.

Ligne 73 : `for element in slice.iter_mut() {` — `.iter_mut()` produit un iterateur d’emprunts mutables. `element` est de type `&mut i32`.

Ligne 74-75 : `if *element == ancienne { *element = nouvelle; }` — Le `*` dereference le `&mut i32` pour acceder a la valeur. C’est necessaire pour la comparaison (`*element`) et l’affectation (`*element = ...`).

Ligne 103 : `let (gauche, droite) = data.split_at_mut(milieu);` — `.split_at_mut(n)` divise une slice mutable en deux slices mutables **disjointes** aux positions `..n` et `n...`. C’est la seule facon d’avoir deux emprunts mutables simultanes : ils doivent etre sur des regions memoire disjointes.

Lignes 119-130 : `decaler_gauche` est generique (`<T: Clone>`). Elle fonctionne avec n’importe quel type clonable. Le trait `Clone` est necessaire car on duplique des valeurs.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.swap(i, j)</code>	Echange deux elements	Deux index <code>usize</code>	<code>()</code>	<code>v.swap(0, 1);</code>
<code>.sort()</code>	Trie la slice	Aucun	<code>()</code>	<code>v.sort();</code> (ordre croissant)

Table 35 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.iter_mut()</code>	Iterateur mutable	Aucun	<code>IterMut<T></code>	Produit <code>&mut T</code>
<code>.split_at_mut(n)</code>	Divise en deux slices mutables	<code>usize</code>	<code>(&mut [T], &mut [T])</code>	Permet 2 emprunts mutables
<code>*ref</code>	Dereference	<code>&mut T</code>	<code>T (mutable)</code>	<code>*r = 5;</code>

Pieges frequents

Erreur	Message du compilateur	Explication
Deux <code>&mut</code> sur des regions qui se chevauchent	<code>cannot borrow as mutable more than once</code>	Meme sur des slices, les regions doivent etre disjointes.
<code>&mut v</code> et <code>&v[..]</code> simultanes	<code>cannot borrow as immutable</code>	<code>&mut</code> exclusif signifie pas d' autre emprunt.
Oublier <code>*element</code> dans la comparaison	Type mismatch <code>&mut i32</code> et <code>i32</code>	Dereferencement necessaire pour acceder a la valeur.

Defi bonus

Implementez `rotation_slice` qui fait tourner les elements d'une slice mutable de `k` positions vers la droite, en $O(n)$ temps et $O(1)$ espace supplementaire.

Exercice 18 : Fonctions qui Retournent des Slices

Objectifs

- Comprendre les durees de vie explicites ('a)
- Retourner des slices depuis des fonctions
- Comprendre l'elision de durees de vie
- Gerer les cas limites (Option, Result)

Enonce

Creez des fonctions qui analysent du texte et retournent des slices avec des durees de vie correctement gerees.

Questions

1. Ecrivez `plus_grand_commencement` qui compare deux `&str` et retourne le plus long debut commun.
2. Ecrivez `extraire_entre` qui retourne le texte entre deux marqueurs.

3. Ecrivez `trouver_prefixe` qui retourne le prefixe commun de deux `&str`.
4. Ecrivez `decouper_en_lignes` qui retourne un `Vec<&str>`.
5. Ecrivez `trim_custom` qui retire des caracteres specifiques au debut et a la fin.
6. Testez avec des cas limites (chaines vides, pas de correspondance).

Indices

- Les durees de vie explicitent la relation entre les parametres et le retour.
- `&'a str` signifie “une reference qui vit au moins aussi longtemps que 'a”.
- L’elision permet d’omettre les durees de vie dans certains cas simples.

Correction complete

```
fn main() {
    // =====
    // 1. Plus grand commencement
    // =====
    println!("== Fonctions retournant des slices ==");
    let s1 = "hello world";
    let s2 = "hello Rust";

    match plus_grand_commencement(s1, s2) {
        Some(debut) => println!("Plus grand debut commun : \"{}\"", debut),
        None => println!("Pas de debut commun"),
    }

    // =====
    // 2. Extraire entre marqueurs
    // =====
    println!("\n--- Extraire entre ---");
    let texte = "Debut [contenu] Fin";
    match extraire_entre(texte, '[', ']') {
        Some(contenu) => println!("Contenu extrait : \"{}\"", contenu),
        None => println!("Marqueurs non trouves"),
    }

    // =====
    // 3. Prefixe commun
    // =====
    println!("\n--- Prefixe commun ---");
    let a = "inter national";
    let b = "inter nette";
    let prefixe = trouver_prefixe(a, b);
    println!("Prefixe commun de \"{}\" et \"{}\" : \"{}\"", a, b, prefixe);

    // =====
    // 4. Decouper en lignes
    // =====
    println!("\n--- Decouper en lignes ---");
    let multiligne = "ligne 1\nligne 2\nligne 3";
    let lignes = decouper_en_lignes(multiligne);
```

```

for (i, ligne) in lignes.iter().enumerate() {
    println!(" Ligne {} : \"{}\"", i, ligne);
}

// =====
// 5. Trim personnalisée
// =====
println!("\n--- Trim custom ---");
let a_trimer = "***Bonjour***";
let nettoye = trim_custom(a_trimer, '*');
println!("Avant : \"{}\"", a_trimer);
println!("Après : \"{}\"", nettoye);

// =====
// 6. Cas limites
// =====
println!("\n--- Cas limites ---");
println!("Vide + texte : \"{}\"", trouver_prefixe("", "abc"));
println!("Pas de commun : \"{}\"", trouver_prefixe("abc", "xyz"));
println!("Identiques : \"{}\"", trouver_prefixe("same", "same"));
}

/// Retourne le plus grand début commun entre deux &str
/// Les durées de vie explicites garantissent que le retour
/// est valide aussi longtemps que les deux entrées
fn plus_grand_commencement<'a>(x: &'a str, y: &'a str) -> Option<&'a str> {
    // Prend la longueur minimale pour éviter les dépassements
    let min_len = if x.len() < y.len() { x.len() } else { y.len() };

    let mut fin = 0;

    // Compare caractère par caractère
    for (_, c1), (_, c2) in x.char_indices().zip(y.char_indices()) {
        if c1 != c2 {
            break;
        }
        // Met à jour fin avec la position en octets après ce caractère
        fin = x.char_indices().next().map(|(i, _)| i).unwrap_or(0);
        // Avance dans les deux chaînes
        // (La logique exacte est plus simple avec as_bytes)
    }

    // Version simplifiée avec as_bytes
    let x_bytes = x.as_bytes();
    let y_bytes = y.as_bytes();
    let mut commun = 0;

    for i in 0..min_len {
        if x_bytes[i] != y_bytes[i] {
            break;
        }
    }

```

```

        commun = i + 1;
    }

    if commun > 0 {
        Some(&x[0..commun])
    } else {
        None
    }
}

/// Extrait le texte entre deux marqueurs
/// Parametres :
///   - s : &str (texte source)
///   - debut : char (marqueur de debut)
///   - fin : char (marqueur de fin)
/// Retourne :
///   - Option<&str> : le contenu entre les marqueurs
fn extraire_entre(s: &str, debut: char, fin: char) -> Option<&str> {
    // Trouve la position du marqueur de debut
    let pos_debut = s.find(debut)?;

    // Trouve la position du marqueur de fin apres le debut
    let apres_debut = &s[(pos_debut + 1)..];
    let pos_fin = apres_debut.find(fin)?;

    // Retourne la slice entre les deux marqueurs
    Some(&apres_debut[0..pos_fin])
}

/// Trouve le prefixe commun de deux &str
/// Parametres :
///   - a, b : les deux chaines
/// Retourne :
///   - &str : le prefixe commun (peut etre vide)
fn trouver_prefixe<'a>(a: &'a str, b: &'a str) -> &'a str {
    let a_bytes = a.as_bytes();
    let b_bytes = b.as_bytes();
    let min_len = a.len().min(b.len());

    let mut commun = 0;
    for i in 0..min_len {
        if a_bytes[i] != b_bytes[i] {
            break;
        }
        commun = i + 1;
    }

    &a[0..commun]
}

/// Decoupe un texte multiligne en vecteur de lignes

```



```

/// Parametre :
///   - s : &str
/// Retourne :
///   - Vec<&str> : chaque ligne est une slice du texte original
fn decouper_en_lignes(s: &str) -> Vec<&str> {
    s.split('\n').collect()
}

/// Retire les caracteres specifiques au debut et a la fin
/// Parametres :
///   - s : &str
///   - c : char a retirer
/// Retourne :
///   - &str : la chaine nettooyee
fn trim_custom(s: &str, c: char) -> &str {
    // Trouve le premier caractere different de c
    let debut = s.find(|ch| ch != c).unwrap_or(s.len());

    // Trouve le dernier caractere different de c
    let fin = s.rfind(|ch| ch != c).map(|i| i + 1).unwrap_or(0);

    if debut <= fin {
        &s[debut..fin]
    } else {
        &s[0..0] // Slice vide
    }
}

```

Explication detaillee ligne par ligne

Ligne 57 : `fn plus_grand_commencement<'a>(x: &'a str, y: &'a str) -> Option<&'a str>` — Declaration explicite des durees de vie. Le `'a` est un parametre de duree de vie. Il garantit que la reference retournee ne vit pas plus longtemps que `x` ou `y`.

Ligne 75-79 : `s.find(debut)?` — L'operateur `?` est un raccourci pour le pattern matching sur `Option`. Si `find` retourne `None`, la fonction retourne immediatement `None`. Sinon, il extrait la valeur.

Ligne 100 : `fn trouver_prefixe<'a>(a: &'a str, b: &'a str) -> &'a str` — Meme principe de durees de vie. Le prefixe retourne est valide aussi longtemps que les deux chaines d'entree.

Ligne 123 : `s.split('\n').collect()` — `.split('\n')` produit un iterateur de `&str`. `.collect()` les rassemble en `Vec<&str>`. Chaque element est une slice du texte original.

Lignes 135-138 : `trim_custom` utilise `.find()` pour le debut et `.rfind()` pour la fin (recherche depuis la droite). Le closure `|ch| ch != c` est un predicat qui cherche le premier caractere different.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
'a	Parametre - de duree de vie		-	&'a str
x.find(c)	Cherche un caractere	char ou closure	Option<usize>	"abc".find('b') -> Some(1)
x.rfind(c)	Cherche depuis la fin	char ou closure	Option<usize>	"abc".rfind('b') -> Some(1)
?	Propagation d' Option/ Result		Sort si None/Err	some_option?
.zip()	Combine deux it- erateurs	Autre iterateur	Zip	(a1,b1), (a2,b2)...
.min()	Minimum de deux valeurs	self, other	Self	5.min(3) -> 3

Pieges frequents

Erreur	Message du compilateur	Explication
Retourner une reference locale	returns a reference to data owned by the current function	Ne retournez jamais &variable_locale.
Durees de vie manquantes	cannot infer appropriate lifetime	Ajoutez les annotations 'a.
? sur Option dans une fonction retournant &str	Type mismatch	Le ? propageate Option, donc la fonction doit retourner Option.

Defi bonus

Ecrivez une fonction `extraire_balises` qui prend un `&str` contenant du XML simple (`<tag>contenu</tag>`) et retourne un `Vec<(&str, &str)>` contenant les paires (`nom_balise`, `contenu`).

Exercice 19 : Vec, String et Ownership Complexe

Objectifs

- Combiner `Vec<String>` avec l'ownership
- Transferer des `String` dans et hors de `Vec`
- Comprendre le mouvement des elements d'un vecteur
- Implementer des structures de donnees simples

Enonce

Creez un programme qui manipule un vecteur de `String` en transferrant l'ownership des chaines.

Questions

1. Creez un `Vec<String>`, ajoutez des elements, puis retirez-les.
2. Ecrivez `deplacer_element` qui transfere un element d'un vec a un autre.
3. Ecrivez `inverser_propriete` qui inverse l'ordre des `String` dans un `Vec`.
4. Ecrivez `filtrer_longueur` qui retourne un nouveau `Vec` avec les `String` d'une certaine longueur.
5. Ecrivez `concatener_tout` qui consomme le `Vec` et retourne une `String`.
6. Montrez la difference entre `.clone()` et le transfert de propriete.

Indices

- `.pop()` retire le dernier element et le retourne (`Option<T>`).
- `.remove(i)` retire l'element a l'index `i` et le retourne.
- `.push()` transfere la propriete dans le vecteur.
- Le transfert de propriete fonctionne element par element.

Correction complete

```
fn main() {  
    // =====  
    // 1. Ajout et retrait d'elements  
    // =====  
    println!("=== Vec<String> et Ownership ===");  
    let mut panier = Vec::new();  
  
    // push() transfere la propriete de la String au Vec  
    panier.push(String::from("Pomme"));  
    panier.push(String::from("Banane"));  
    panier.push(String::from("Cerise"));  
    println!("Panier : {:?}", panier);  
  
    // pop() transfere la propriete du Vec vers la variable  
    if let Some(fruit) = panier.pop() {  
        println!("Retire du panier : \"{}\"", fruit);  
    }  
    println!("Panier apres pop : {:?}", panier);  
  
    // remove() transfere un element specifique  
    let retire = panier.remove(0);  
    println!("Retire a l'index 0 : \"{}\"", retire);  
    println!("Panier apres remove : {:?}", panier);  
  
    // =====  
    // 2. Transfert entre vecteurs  
    // =====  
    println!("\n--- Deplacer element ---");  
    let mut source = vec![  
        String::from("A"),
```

```

        String::from("B"),
        String::from("C"),
    ];
    let mut destination = Vec::new();

    println!("Source avant : {:?}", source);
    deplacer_element(&mut source, &mut destination, 1);
    println!("Source apres : {:?}", source);
    println!("Destination : {:?}", destination);

    // =====
    // 3. Inverser l'ordre par transfert
    // =====
    println!("\n--- Inverser propriete ---");
    let mut mots = vec![
        String::from("premier"),
        String::from("deuxieme"),
        String::from("troisieme"),
    ];
    println!("Avant : {:?}", mots);
    inverser_propriete(&mut mots);
    println!("Apres : {:?}", mots);

    // =====
    // 4. Filtrer par longueur
    // =====
    println!("\n--- Filtrer longueur ---");
    let mots = vec![
        String::from("Le"),
        String::from("langage"),
        String::from("Rust"),
        String::from("est"),
        String::from("formidable"),
    ];
    let longs = filtrer_longueur(&mots, 5);
    println!("Mots originaux : {:?}", mots);
    println!("Mots de >= 5 caracteres : {:?}", longs);

    // =====
    // 5. Concatenation consommant
    // =====
    println!("\n--- Concatener tout ---");
    let fragments = vec![
        String::from("Hello"),
        String::from(" "),
        String::from("World"),
        String::from("!"),
    ];
    let phrase = concatener_tout(fragments);
    // fragments n'est plus utilisable ici !
    println!("Phrase concaténée : \"{}\"", phrase);

```

```

// =====
// 6. Clone vs Transfert
// =====

println!("\n--- Clone vs Transfert ---");
let original = String::from("Important");

// Transfert : original n'est plus utilisable
let transfert = original;
println!("Transfert : \"{}\"", transfert);
// println!("Original : {}", original); // ERREUR !

// Clone : les deux sont utilisables
let clone_source = String::from("Clonable");
let clone_copie = clone_source.clone();
println!("Source : \"{}\"", clone_source);
println!("Copie : \"{}\"", clone_copie);
}

/// Deplace un element d'un vecteur vers un autre
/// Parametres :
/// - src : &mut Vec<String> (source)
/// - dst : &mut Vec<String> (destination)
/// - index : usize (index de l'element a deplacer)
fn deplacer_element(src: &mut Vec<String>, dst: &mut Vec<String>, index: usize) {
    if index >= src.len() {
        return;
    }

    // remove() retire l'element et transfere la propriete
    let element = src.remove(index);

    // push() transfere la propriete au vecteur destination
    dst.push(element);
}

/// Inverse l'ordre des String dans un Vec en utilisant pop/push
/// Parametre :
/// - v : &mut Vec<String>
fn inverser_propriete(v: &mut Vec<String>) {
    let mut temp = Vec::new();

    // Vide le vecteur original dans temp (ordre inverse)
    while let Some(element) = v.pop() {
        temp.push(element);
    }

    // Transfere de temp vers v
    for element in temp {
        v.push(element);
    }
}

```

```

}

/// Filtre les String par longueur minimale
/// Parametres :
///   - v : &Vec<String> (emprunt immuable)
///   - min_len : usize (longueur minimale)
/// Retourne :
///   - Vec<String> avec les String qui satisfont la condition
fn filtrer_longueur(v: &Vec<String>, min_len: usize) -> Vec<String> {
    let mut resultat = Vec::new();

    for s in v.iter() {
        if s.len() >= min_len {
            // clone() car on ne peut pas transférer depuis un emprunt immuable
            resultat.push(s.clone());
        }
    }

    resultat
}

/// Consomme un Vec<String> et retourne une String concaténée
/// Parametre :
///   - v : Vec<String> (ownership transférée)
/// Retourne :
///   - String : la concaténation de tous les éléments
fn concatener_tout(v: Vec<String>) -> String {
    let mut resultat = String::new();

    for s in v {
        // s est transférée dans la boucle (ownership de chaque élément)
        resultat.push_str(&s);
        // s est dropée ici
    }

    resultat
}

```

Explication détaillée ligne par ligne

Ligne 11-13 : `panier.push(String::from("Pomme"));` — La `String` est créée, puis sa **propriété est transférée** au vecteur via `push`. Le vecteur devient propriétaire de la `String`.

Ligne 17 : `if let Some(fruit) = panier.pop() {` — `.pop()` retire le dernier élément et **transfère sa propriété** à la variable `fruit`. Le vecteur n'est plus propriétaire de cette `String`.

Ligne 22 : `let retire = panier.remove(0);` — `.remove(index)` retire l'élément à l'index spécifié et transfère sa propriété. Les éléments suivants sont décalés vers la gauche.

Lignes 50-52 : `deplacer_element` utilise `.remove()` pour sortir l'élément du vecteur source (transfert de propriété à la variable locale `element`), puis `.push()` pour transférer au vecteur destination.

Lignes 70-78 : `inverser_propriete` utilise un vecteur temporaire. `.pop()` retire les elements du vecteur original (propriete transferee a `element`, puis a `temp`). Ensuite, une boucle `for element in temp` consomme le vecteur temporaire (chaque element est transfere a `v`).

Ligne 95 : `resultat.push(s.clone());` — Comme `filtrer_longueur` prend un `&Vec<String>` (emprunt immuable), on ne peut pas transferer les elements. On utilise `.clone()` pour creer une copie.

Lignes 108-115 : `concatener_tout` consomme le vecteur. La boucle `for s in v` prend possession de chaque element du vecteur. `s` est une `String` owned qui est passee a `push_str` (via `&s`, un emprunt temporaire).

Bibliothèques et fonctions utilisées

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.pop()</code>	Retire le dernier element	Aucun	<code>Option<T></code>	Transfere la propriete
<code>.remove(i)</code>	Retire a l' index <code>i</code>	<code>usize</code>	<code>T</code>	Decale les elements
<code>.push(v)</code>	Ajoute a la fin	<code>T</code>	<code>()</code>	Transfere la propriete
<code>.clone()</code>	Copie profonde	Aucun	<code>Self</code>	Duplique les donnees
<code>if let Some(x) = opt</code>	Pattern matching	<code>Option</code>	Execute si <code>Some</code>	Extrait la valeur
<code>while let Some(x) = opt</code>	Boucle conditionnelle	<code>Option</code>	Itere tant que <code>Some</code>	Consomme progressivement

Pieges frequents

Erreur	Message du compilateur	Explication
<code>v.iter()</code> puis modification	<code>cannot borrow as mutable</code>	L' iterateur emprunte le vecteur.
<code>s</code> utilisee apres <code>push(s)</code>	<code>value moved here</code>	La propriete a ete transferee au vecteur.
<code>.remove()</code> hors bornes	<code>index out of bounds</code>	Verifiez que l' index est valide.
<code>for s in &v</code> avec transfert	<code>cannot move out of borrowed content</code>	Utilisez <code>for s in v</code> (consommer) ou clonez.

Defi bonus

Implementez une fonction `reorganiser` qui prend un `Vec<String>` et retourne deux `Vec<String>` : un avec les chaines contenant uniquement des caracteres alphabetiques, et un avec les autres. La fonction doit transferer les elements (pas les cloner).

Exercice 20 : Programme Complet — Editeur de Texte Simplifie

Objectifs

- Combiner tous les concepts : ownership, borrowing, slices, Vec, String
- Créer un programme coherent et fonctionnel
- Utiliser les references mutables et immuables de maniere appropriée
- Implementer une mini-application interactive

Enonce

Creez un editeur de texte simplifie en ligne de commande qui peut stocker des lignes de texte, les afficher, les modifier, et les analyser.

Questions

1. Implementez `ajouter_ligne` qui ajoute une ligne au document.
2. Implementez `afficher_document` qui affiche toutes les lignes avec numeros.
3. Implementez `modifier_ligne` qui remplace une ligne specifique.
4. Implementez `supprimer_ligne` qui retire une ligne.
5. Implementez `rechercher` qui trouve toutes les lignes contenant un mot.
6. Implementez `statistiques` qui compte les lignes, mots et caracteres.
7. Implementez `inverser_lignes` qui inverse l'ordre des lignes.

Indices

- Utilisez `Vec<String>` pour stocker les lignes.
- Chaque ligne est une `String` owned par le vecteur.
- Utilisez `&[String]` ou `&Vec<String>` pour les fonctions de lecture.
- Utilisez `&mut Vec<String>` pour les fonctions de modification.

Correction complete

```
use std::io::{self, Write};

fn main() {
    let mut document: Vec<String> = Vec::new();

    println!("=== Editeur de Texte Simplifie ===");
    println!("Commandes : ajouter, afficher, modifier, supprimer,");
    println!("          rechercher, stats, inverser, quitter\n");

    loop {
        print!("> ");
        io::stdout().flush().unwrap();

        let mut commande = String::new();
        io::stdin().read_line(&mut commande).unwrap();

        let parties: Vec<&str> = commande.trim().split_whitespace().collect();
        if parties.is_empty() {
            continue;
        }
    }
}
```



```

}

match parties[0] {
  "ajouter" => {
    if parties.len() > 1 {
      let texte = parties[1..].join(" ");
      ajouter_ligne(&mut document, texte);
      println!("Ligne ajoutée.");
    } else {
      println!("Usage : ajouter <texte>");
    }
  }
  "afficher" => {
    afficher_document(&document);
  }
  "modifier" => {
    if parties.len() >= 3 {
      if let Ok(num) = parties[1].parse::<usize>() {
        let texte = parties[2..].join(" ");
        modifier_ligne(&mut document, num, texte);
      } else {
        println!("Numero de ligne invalide.");
      }
    } else {
      println!("Usage : modifier <numero> <texte>");
    }
  }
  "supprimer" => {
    if parties.len() >= 2 {
      if let Ok(num) = parties[1].parse::<usize>() {
        supprimer_ligne(&mut document, num);
      } else {
        println!("Numero de ligne invalide.");
      }
    } else {
      println!("Usage : supprimer <numero>");
    }
  }
  "rechercher" => {
    if parties.len() >= 2 {
      let mot = parties[1];
      rechercher(&document, mot);
    } else {
      println!("Usage : rechercher <mot>");
    }
  }
  "stats" => {
    statistiques(&document);
  }
  "inverser" => {
    inverser_lignes(&mut document);
  }
}

```

```

        println!("Lignes inversees.");
    }
    "quitter" => {
        println!("Au revoir !");
        break;
    }
    _ => {
        println!("Commande inconnue.");
    }
}
}

/// Ajoute une ligne au document
/// Parametres :
/// - doc : &mut Vec<String> (document modifiable)
/// - ligne : String (la ligne a ajouter, ownership transfere)
fn ajouter_ligne(doc: &mut Vec<String>, ligne: String) {
    doc.push(ligne);
}

/// Affiche le document avec numeros de ligne
/// Parametre :
/// - doc : &Vec<String> (emprunt immuable)
fn afficher_document(doc: &Vec<String>) {
    if doc.is_empty() {
        println!("(Document vide)");
        return;
    }

    println!("--- Document ---");
    for (i, ligne) in doc.iter().enumerate() {
        println!("{:>3} | {}", i + 1, ligne);
    }
    println!("-----");
}

/// Modifie une ligne existante
/// Parametres :
/// - doc : &mut Vec<String>
/// - numero : usize (numero de ligne, 1-indexe)
/// - nouveau_texte : String
fn modifier_ligne(doc: &mut Vec<String>, numero: usize, nouveau_texte: String) {
    let index = numero.saturating_sub(1);

    if index < doc.len() {
        doc[index] = nouveau_texte;
        println!("Ligne {} modifiee.", numero);
    } else {
        println!("Ligne {} inexistante.", numero);
    }
}

```

```

}

/// Supprime une ligne du document
/// Parametres :
/// - doc : &mut Vec<String>
/// - numero : usize (numero de ligne, 1-indexe)
fn supprimer_ligne(doc: &mut Vec<String>, numero: usize) {
    let index = numero.saturating_sub(1);

    if index < doc.len() {
        doc.remove(index);
        println!("Ligne {} supprimee.", numero);
    } else {
        println!("Ligne {} inexistante.", numero);
    }
}

/// Recherche un mot dans toutes les lignes
/// Parametres :
/// - doc : &Vec<String> (emprunt immuable)
/// - mot : &str (mot a chercher)
fn rechercher(doc: &Vec<String>, mot: &str) {
    println!("--- Resultats pour \"{}\" ---", mot);
    let mut trouve = false;

    for (i, ligne) in doc.iter().enumerate() {
        if ligne.contains(mot) {
            println!("{:>3} | {}", i + 1, ligne);
            trouve = true;
        }
    }

    if !trouve {
        println!("Aucun resultat.");
    }
    println!("-----");
}

/// Affiche les statistiques du document
/// Parametre :
/// - doc : &Vec<String>
fn statistiques(doc: &Vec<String>) {
    let nb_lignes = doc.len();
    let nb_caracteres: usize = doc.iter().map(|s| s.len()).sum();
    let nb_mots: usize = doc.iter()
        .map(|s| s.split_whitespace().count())
        .sum();

    println!("--- Statistiques ---");
    println!("Lignes : {}", nb_lignes);
    println!("Mots : {}", nb_mots);
}

```

```
println!("Caracteres : {}", nb_caracteres);
println!("-----");
}

/// Inverse l'ordre des lignes
/// Parametre :
/// - doc : &mut Vec<String>
fn inverser_lignes(doc: &mut Vec<String>) {
    doc.reverse();
}
```

Explication detaillee ligne par ligne

Lignes 1-2 : use std::io::{self, Write}; — Import des modules necessaires pour l’entree/sortie. Write est necessaire pour la methode .flush() qui force l’affichage du prompt.

Ligne 20-21 : let parties: Vec<&str> = commande.trim().split_whitespace().collect(); — Decoupe la commande en mots. Chaque mot est une &str qui fait reference a la String commande.

Ligne 30 : let texte = parties[1..].join(" "); — .join() rassemble les slices en une String separee par des espaces. La propriete de cette String est transferee a ajouter_ligne.

Ligne 78 : fn ajouter_ligne(doc: &mut Vec<String>, ligne: String) { — La fonction prend une reference mutable sur le document (pour pouvoir le modifier) et une String owned (pour la propriete de la nouvelle ligne).

Ligne 79 : doc.push(ligne); — Transfert de propriete de ligne au vecteur. La String est stockee dans le vecteur.

Ligne 102 : doc[index] = nouveau_texte; — Remplacement d’une ligne. L’ancienne String est automatiquement detruite (drop), et la nouvelle prend sa place.

Ligne 117 : doc.remove(index); — Retire la ligne et la retourne. Comme on n’assigne pas le resultat, la String retiree est immediatement detruite.

Ligne 140 : let nb_caracteres: usize = doc.iter().map(|s| s.len()).sum(); — Chaine d’iterateurs : .iter() emprunte, .map() transforme chaque &String en sa longueur, .sum() additionne.

Ligne 157 : doc.reverse(); — Methode standard des Vec qui inverse l’ordre des elements en place. Les String ne sont pas deplacees en memoire, seuls les pointeurs sont echanges.

Bibliotheques et fonctions utilisees

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
.split_whitespace()	Separe par espaces	Aucun	SplitWhitespace	"a b".split_whitespace()
.join(sep)	Rassemble &str avec separeateur		String	["a","b"].join(", ")
.parse::<T>()	Convertit en type	Aucun	Result<T, E>	"42".parse::<i32>()

Table 41 – continued

Fonction	Role	Parametres	Valeur retournee	Exemple supplementaire
<code>.saturating_sub(8)</code>	sans un- derflow	<code>usize</code>	<code>usize</code>	<code>0.saturating_sub(1) -> 0</code>
<code>.contains(sub)</code>	Teste sous- chaine	<code>&str</code>	<code>bool</code>	<code>"abc".contains("b")</code>
<code>.reverse()</code>	Inverse le vecteur	Aucun	<code>()</code>	Modifie en place
<code>.flush()</code>	Force l' affichage	Aucun	<code>Result</code>	Pour les prompts interactifs

Pieges frequents

Erreur	Message du compilateur	Explication
<code>parties[1..]</code> sans <code>join</code>	Type <code>&[&str]</code> , pas <code>String</code>	Il faut rassembler les slices en <code>String</code> .
<code>doc emprunte</code> <code>mutably</code> et <code>immutably</code>	<code>cannot borrow as mutable</code>	Verifiez les scopes des emprunts.
Oublier <code>.saturat- ing_sub(1)</code>	Panique si <code>numero = 0</code>	<code>0 - 1</code> en <code>usize</code> panique. Utilisez <code>saturating_sub</code> .

Defi bonus

Ajoutez une commande `sauvegarder <fichier>` qui ecrit le document dans un fichier, et `charger <fichier>` qui lit un fichier. Utilisez `std::fs::File` et `std::io::{BufRead, BufReader}`.

Partie 2 : 5 Projets Complets

Projet 1 : Gestionnaire de Notes

Presentation

Ce projet consiste a creer un systeme de gestion de notes pour une petite classe d'etudiants. L'application permet d'ajouter des etudiants, d'attribuer des notes, de calculer des moyennes, et de generer des statistiques.

Cahier des charges

Fonctionnalite	Description
Ajouter un etudiant	Enregistrer un nouvel etudiant avec son nom
Ajouter une note	Attribuer une note a un etudiant pour une matiere
Calculer la moyenne	Calculer la moyenne d' un etudiant
Statistiques globales	Afficher les moyennes de la classe
Rechercher	Trouver un etudiant par son nom
Classement	Afficher le classement des etudiants

Concepts Rust utilises

Concept	Utilisation
Vec	Stockage des etudiants et des notes
String	Noms des etudiants et matieres
Structs	Definition des types Etudiant et Note
Ownership	Transfert des noms et notes
Borrowing	Fonctions d' affichage et de calcul
References	Passage des etudiants aux fonctions

Architecture

```
gestionnaire_notes/
|-- main()           Point d'entree
|-- struct Etudiant  Nom + Vec<Note>
|-- struct Note      Matiere + valeur
|-- ajouter_etudiant() Ajoute a la liste
|-- ajouter_note()    Attribue une note
|-- calculer_moyenne() Calcule la moyenne
|-- afficher_statistiques() Stats globales
|-- rechercher_etudiant() Recherche par nom
|-- classer_etudiants() Classement par moyenne
```

Etapes de realisation

1. Definir les structures **Note** et **Etudiant**
2. Implementer la fonction d'ajout d'etudiant
3. Implementer l'attribution de notes
4. Implementer le calcul de moyenne
5. Implementer les statistiques globales
6. Implementer la recherche
7. Implementer le classement
8. Creer le menu interactif

Code complet

```
use std::io::{self, Write};
```

```
// =====
// STRUCTURES DE DONNEES
// =====

/// Represente une note dans une matiere
/// Champs :
///   - matiere : String (nom de la matiere)
///   - valeur : f64 (note sur 20)
struct Note {
    matiere: String,
    valeur: f64,
}

/// Represente un etudiant avec ses notes
/// Champs :
///   - nom : String (nom de l'etudiant)
///   - notes : Vec<Note> (liste des notes)
struct Etudiant {
    nom: String,
    notes: Vec<Note>,
}

/// Implementation des methodes pour Etudiant
impl Etudiant {
    /// Cree un nouvel etudiant (constructeur)
    /// Parametre :
    ///   - nom : String
    /// Retourne :
    ///   - Etudiant
    fn nouveau(nom: String) -> Etudiant {
        Etudiant {
            nom,
            notes: Vec::new(),
        }
    }

    /// Ajoute une note a l'etudiant
    /// Parametres :
    ///   - matiere : String
    ///   - valeur : f64
    fn ajouter_note(&mut self, matiere: String, valeur: f64) {
        self.notes.push(Note { matiere, valeur });
    }

    /// Calcule la moyenne de l'etudiant
    /// Retourne :
    ///   - Option<f64> : Some(moyenne) ou None si pas de notes
    fn moyenne(&self) -> Option<f64> {
        if self.notes.is_empty() {
            return None;
        }
    }
}
```

```

        let somme: f64 = self.notes.iter().map(|n| n.valeur).sum();
        Some(somme / self.notes.len() as f64)
    }
}

// =====
// FONCTIONS DU PROGRAMME
// =====

/// Ajoute un etudiant a la liste
/// Parametres :
///   - etudiants : &mut Vec<Etudiant>
///   - nom : String
fn ajouter_etudiant(etudiants: &mut Vec<Etudiant>, nom: String) {
    etudiants.push(Etudiant::nouveau(nom));
}

/// Trouve un etudiant par son nom (reference mutable)
/// Parametres :
///   - etudiants : &mut Vec<Etudiant>
///   - nom : &str
/// Retourne :
///   - Option<&mut Etudiant>
fn trouver_etudiant_mut<'a>(
    etudiants: &'a mut Vec<Etudiant>,
    nom: &str,
) -> Option<&'a mut Etudiant> {
    etudiants.iter_mut().find(|e| e.nom == nom)
}

/// Trouve un etudiant par son nom (reference immuable)
/// Parametres :
///   - etudiants : &Vec<Etudiant>
///   - nom : &str
/// Retourne :
///   - Option<&Etudiant>
fn trouver_etudiant<'a>(etudiants: &'a Vec<Etudiant>, nom: &str) -> Option<&'a Etudiant> {
    etudiants.iter().find(|e| e.nom == nom)
}

/// Affiche les statistiques globales
/// Parametre :
///   - etudiants : &Vec<Etudiant>
fn afficher_statistiques(etudiants: &Vec<Etudiant>) {
    if etudiants.is_empty() {
        println!("Aucun etudiant enregistre.");
        return;
    }

    println!("\n=== Statistiques Globales ===");

```



```

println!("Nombre d'etudiants : {}", etudiants.len());

// Moyenne de la classe
let mut total_classe: f64 = 0.0;
let mut nb_moyennes: usize = 0;

for etudiant in etudiants {
    if let Some(moy) = etudiant.moyenne() {
        total_classe += moy;
        nb_moyennes += 1;
    }
}

if nb_moyennes > 0 {
    println!("Moyenne de la classe : {:.2}/20", total_classe / nb_moyennes as f64);
}

// Details par etudiant
println!("\n--- Details ---");
for etudiant in etudiants {
    match etudiant.moyenne() {
        Some(moy) => println!(" {:<20} : {:>5.2}/20 ({} notes)",
                               etudiant.nom, moy, etudiant.notes.len()),
        None => println!(" {:<20} : aucune note", etudiant.nom),
    }
}
println!("=====\n");
}

/// Affiche le classement des etudiants
/// Parametre :
/// - etudiants : &Vec<Etudiant>
fn afficher_classement(etudiants: &Vec<Etudiant>) {
    let mut avec_moyenne: Vec<(&str, f64)> = Vec::new();

    for e in etudiants {
        if let Some(moy) = e.moyenne() {
            avec_moyenne.push((&e.nom, moy));
        }
    }

    // Tri par moyenne decroissante
    avec_moyenne.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap());

    println!("\n=== Classement ===");
    for (i, (nom, moy)) in avec_moyenne.iter().enumerate() {
        println!(" {} {:<20} : {:>5.2}/20", i + 1, nom, moy);
    }
    println!("=====\n");
}

```

```

/// Point d'entree
fn main() {
    let mut etudiants: Vec<Etudiant> = Vec::new();

    // Donnees de demonstration
    println!("=== Gestionnaire de Notes ===\n");

    ajouter_etudiant(&mut etudiants, String::from("Alice Martin"));
    ajouter_etudiant(&mut etudiants, String::from("Bob Dupont"));
    ajouter_etudiant(&mut etudiants, String::from("Charlie Durand"));

    // Ajout de notes
    if let Some(e) = trouver_etudiant_mut(&mut etudiants, "Alice Martin") {
        e.ajouter_note(String::from("Mathematiques"), 16.5);
        e.ajouter_note(String::from("Physique"), 14.0);
        e.ajouter_note(String::from("Informatique"), 18.0);
    }

    if let Some(e) = trouver_etudiant_mut(&mut etudiants, "Bob Dupont") {
        e.ajouter_note(String::from("Mathematiques"), 12.0);
        e.ajouter_note(String::from("Physique"), 10.5);
        e.ajouter_note(String::from("Informatique"), 13.0);
    }

    if let Some(e) = trouver_etudiant_mut(&mut etudiants, "Charlie Durand") {
        e.ajouter_note(String::from("Mathematiques"), 19.0);
        e.ajouter_note(String::from("Physique"), 17.5);
        e.ajouter_note(String::from("Informatique"), 19.5);
    }

    // Affichage des statistiques
    afficher_statistiques(&etudiants);

    // Recherche
    println!("--- Recherche ---");
    match trouver_etudiant(&etudiants, "Alice Martin") {
        Some(e) => {
            println!("Etudiant trouve : {}", e.nom);
            println!("Notes :");
            for note in &e.notes {
                println!("  {} : {:.1}", note.matiere, note.valeur);
            }
            if let Some(moy) = e.moyenne() {
                println!("Moyenne : {:.2}/20", moy);
            }
        }
        None => println!("Etudiant non trouve."),
    }

    // Classement
    afficher_classement(&etudiants);
}

```

```
}
```

Explication detaillee

Lignes 16-27 : Les structs `Note` et `Etudiant` definissent les structures de donnees. `Note` possede une `String` pour la matiere (`ownership`). `Etudiant` possede un nom (`String`) et un vecteur de `Note` (`ownership`).

Lignes 30-56 : Le bloc `impl Etudiant` definit des **methodes** associees au type. `nouveau` est un constructeur. `ajouter_note` prend `&mut self` car elle modifie l'etudiant. `moyenne` prend `&self` car elle ne fait que lire.

Ligne 66-72 : `trouver_etudiant_mut` retourne `Option<&mut Etudiant>`. La duree de vie 'a lie la reference retournee au vecteur d'entree.

Lignes 91-120 : `afficher_statistiques` utilise uniquement des references immuables (`&Vec<Etudiant>`). Elle calcule la moyenne de la classe en agregant les moyennes individuelles.

Lignes 123-140 : `afficher_classement` cree un vecteur temporaire de tuples (`nom`, `moyenne`), le trie avec `.sort_by()`, puis l'affiche. Le closure de tri utilise `partial_cmp` car les `f64` ne sont pas totalement ordonnes (`NaN`).

Fonctions et bibliotheques utilisees

Fonction	Role	Parametres	Valeur retournee
<code>.iter_mut().find()</code>	Cherche avec critere	Closure	<code>Option<&mut T></code>
<code>.sort_by()</code>	Trie avec comparateur	Closure	<code>()</code>
<code>partial_cmp()</code>	Comparaison partielle	<code>&f64</code>	<code>Option<Ordering></code>
<code>.push(struct { ... })</code>	Ajoute une instance	Champs	<code>()</code>

Erreurs frequentes

Erreur	Cause
<code>cannot borrow as mutable value moved</code>	Emprunt immutable actif pendant modification
<code>partial_cmp needed</code>	Transfert de propriete sans clone <code>f64</code> ne implemente pas <code>Ord</code>

Ameliorations possibles

1. Persistance des donnees dans un fichier CSV
2. Gestion des coefficients par matiere
3. Interface utilisateur avec menu interactif
4. Export des resultats en format PDF
5. Gestion des classes multiples
6. Historique des notes par trimestre

7. Generation de bulletins scolaires
8. Calcul des ecart-types
9. Ajout de commentaires par note
10. Suppression d'étudiants avec confirmation

Defi expert

Ajoutez une gestion complete des matieres avec coefficients, supportez la persistance JSON avec serde, et implementez une recherche full-text sur les noms d'étudiants.

Projet 2 : Gestionnaire de Taches Todo CLI

Presentation

Un gestionnaire de taches en ligne de commande qui permet d'organiser ses taches quotidiennes. L'application supporte l'ajout, la completion, le filtrage, et la priorisation des taches.

Cahier des charges

Fonctionnalite	Description
Ajouter une tache	Description + priorite
Lister les taches	Toutes ou filtrees par statut
Marquer comme faite	Changer le statut d' une tache
Supprimer une tache	Retirer une tache
Modifier priorite	Changer l' urgence d' une tache
Rechercher	Chercher dans les descriptions
Statistiques	Taux de completion

Concepts Rust utilises

Concept	Utilisation
Vec	Stockage des taches
String	Descriptions des taches
Enum	Statut (A faire / Fait) et priorite
Struct	Type Tache
Ownership	Transfert des descriptions
Borrowing	Affichage et modification

Architecture

```
todo_cli/  
|-- enum Statut      Afaire / Fait  
|-- enum Priorite    Basse / Moyenne / Haute  
|-- struct Tache     description + statut + priorite  
|-- ajouter_tache()  Cree et ajoute  
|-- lister_taches()  Affiche avec filtre  
|-- completer_tache() Marque comme faite
```

```
|-- supprimer_tache() Retire une tache
|-- modifier_priorite() Change la priorite
|-- rechercher_taches() Cherche par mot-cle
|-- statistiques() Taux de completion
```

Code complet

```
use std::io::{self, Write};

// =====
// ENUMERATIONS
// =====

/// Statut d'une tache
#[derive(Debug, Clone, PartialEq)]
enum Statut {
    Afaire,
    Fait,
}

/// Niveau de priorite
#[derive(Debug, Clone, PartialEq)]
enum Priorite {
    Basse,
    Moyenne,
    Haute,
}

impl Priorite {
    /// Retourne la priorite sous forme de chaine pour l'affichage
    fn afficher(&self) -> &str {
        match self {
            Priorite::Basse => "BASSE",
            Priorite::Moyenne => "MOYENNE",
            Priorite::Haute => "HAUTE",
        }
    }

    /// Ordre de tri : Haute = 0, Moyenne = 1, Basse = 2
    fn ordre(&self) -> usize {
        match self {
            Priorite::Haute => 0,
            Priorite::Moyenne => 1,
            Priorite::Basse => 2,
        }
    }
}

// =====
// STRUCTURE TACHE
// =====
```

```

/// Represente une tache a effectuer
#[derive(Debug)]
struct Tache {
    id: usize,
    description: String,
    statut: Statut,
    priorite: Priorite,
}

impl Tache {
    /// Cree une nouvelle tache
    fn nouvelle(id: usize, description: String, priorite: Priorite) -> Tache {
        Tache {
            id,
            description,
            statut: Statut::Afaire,
            priorite,
        }
    }

    /// Marque la tache comme faite
    fn completer(&mut self) {
        self.statut = Statut::Fait;
    }

    /// Change la priorite
    fn set_priorite(&mut self, priorite: Priorite) {
        self.priorite = priorite;
    }

    /// Affiche la tache formatee
    fn afficher(&self) {
        let statut_icone = match self.statut {
            Statut::Fait => "[X]",
            Statut::Afaire => "[ ]",
        };
        println!(" {:>4} | {} | {:<7} | {} ",
            self.id, statut_icone, self.priorite.afficher(), self.description);
    }
}

// =====
// FONCTIONS
// =====

/// Ajoute une tache a la liste
fn ajouter_tache(taches: &mut Vec<Tache>, description: String, priorite: Priorite, next_id: &mut usize)
-> {
    taches.push(Tache::nouvelle(*next_id, description, priorite));
    *next_id += 1;
}

```

```

}

/// Liste les taches avec filtre optionnel
fn lister_taches(taches: &Vec<Tache>, filtre_statut: Option<&Statut>) {
    println!("\n=== Liste des taches ===");
    println!("{}", "{:>4} | {:3} | {:<7} | {}", "ID", "", "PRIORITE", "DESCRIPTION");
    println!("{}", "-----");

    let mut compteur = 0;

    for tache in taches {
        if let Some(statut) = filtre_statut {
            if &tache.statut != statut {
                continue;
            }
        }
        tache.afficher();
        compteur += 1;
    }

    if compteur == 0 {
        println!(" (Aucune tache)");
    }
    println!("=====\n");
}

/// Trouve une tache par son ID (mutable)
fn trouver_tache_mut<'a>(taches: &'a mut Vec<Tache>, id: usize) -> Option<&'a mut Tache> {
    taches.iter_mut().find(|t| t.id == id)
}

/// Trouve une tache par son ID (immutable)
fn trouver_tache<'a>(taches: &'a Vec<Tache>, id: usize) -> Option<&'a Tache> {
    taches.iter().find(|t| t.id == id)
}

/// Supprime une tache par son ID
fn supprimer_tache(taches: &mut Vec<Tache>, id: usize) -> bool {
    let position = taches.iter().position(|t| t.id == id);
    if let Some(pos) = position {
        taches.remove(pos);
        true
    } else {
        false
    }
}

/// Recherche des taches par mot-cle
fn rechercher_taches<'a>(taches: &'a Vec<Tache>, mot: &str) -> Vec<&'a Tache> {
    taches.iter()
        .filter(|t| t.description.contains(mot))

```

```

        .collect()
    }

    /// Affiche les statistiques
    fn statistiques(taches: &Vec<Tache>) {
        let total = taches.len();
        let faites = taches.iter().filter(|t| t.statut == Statut::Fait).count();
        let a_faire = total - faites;

        println!("\n=== Statistiques ===");
        println!("  Total : {}", total);
        println!("  Faites : {}", faites);
        println!("  A faire : {}", a_faire);

        if total > 0 {
            println!("  Taux de completion : {:.1}%", (faites as f64 / total as f64) * 100.0);
        }
        println!("=====\n");
    }

    /// Trie les taches par priorite
    fn trier_par_priorite(taches: &mut Vec<Tache>) {
        taches.sort_by_key(|t| t.priorite.ordre());
    }

    // =====
    // MAIN
    // =====

    fn main() {
        let mut taches: Vec<Tache> = Vec::new();
        let mut next_id: usize = 1;

        println!("=== Gestionnaire de Taches ===\n");

        // Taches de demonstration
        ajouter_tache(&mut taches, String::from("Faire les courses"), Priorite::Moyenne, &mut next_id);
        ajouter_tache(&mut taches, String::from("Preparer le rapport"), Priorite::Haute, &mut next_id);
        ajouter_tache(&mut taches, String::from("Repondre aux emails"), Priorite::Basse, &mut next_id);
        ajouter_tache(&mut taches, String::from("Reunion d'equipe"), Priorite::Haute, &mut next_id);

        // Completer une tache
        if let Some(tache) = trouver_tache_mut(&mut taches, 1) {
            tache.completer();
        }

        // Affichage
        lister_taches(&taches, None);

        // Filtrer par statut
        println!("--- Taches a faire ---");
    }

```



```
lister_taches(&taches, Some(&Statut::Afaire));

// Recherche
println!("--- Recherche : 'rapport' ---");
let resultats = rechercher_taches(&taches, "rapport");
for t in resultats {
    t.afficher();
}

// Statistiques
statistiques(&taches);

// Tri par priorite
trier_par_priorite(&mut taches);
println!("--- Apres tri par priorite ---");
lister_taches(&taches, None);

// Suppression
if supprimer_tache(&mut taches, 2) {
    println!("Tache 2 supprimee.");
}
lister_taches(&taches, None);
}
```

Explication detaillee

Lignes 12-30 : Les enums Statut et Priorite representent les etats discrets. `#[derive(Debug, Clone, PartialEq)]` genere automatiquement les implementations des traits Debug (affichage), Clone (copie), et PartialEq (comparaison).

Lignes 33-47 : `impl Priorite` associe des methodes aux variantes de l'enum. `afficher` retourne une `&str` static (chaîne literal). `ordre` retourne un nombre pour le tri.

Lignes 57-86 : `impl Tache` definit les methodes de la struct. `completer` et `set_priorite` prennent `&mut self`. `afficher` prend `&self` et utilise le pattern matching sur le statut.

Ligne 122 : `rechercher_taches` retourne `Vec<&Tache>` — un vecteur de references sur les taches originales. Ces references empruntent le vecteur `taches`.

Ligne 140 : `.sort_by_key(|t| t.priorite.ordre())` trie le vecteur en place selon la cle retournee par le closure.

Fonctions et bibliotheques

Fonction	Role
<code>#[derive(...)]</code>	Derivation automatique de traits
<code>.position()</code>	Trouve l' index d' un element
<code>.sort_by_key()</code>	Trie selon une cle extraite
<code>.filter()</code>	Filtre un iterateur
enum	Type somme pour les etats discrets

Erreurs frequentes

Erreur	Cause
Match non exhaustif	Oublier une variante d' <code>enum</code>
<code>&mut</code> sur <code>enum</code>	Pattern matching complexe
<code>.position()</code> puis <code>.remove()</code>	Decalage des indices

Ameliorations possibles

1. Persistence dans un fichier
2. Dates d'echeance
3. Categories de taches
4. Sous-taches
5. Export CSV
6. Interface avec couleurs (crate `colored`)
7. Historique des modifications
8. Taches recurrentes
9. Notifications
10. Synchronisation cloud

Defi expert

Implementez un systeme de tags (labels), des dates d'echeance avec tri chronologique, et une sauvegarde automatique avec `serde_json`.

Projet 3 : Gestionnaire de Bibliotheque

Presentation

Systeme de gestion pour une bibliotheque qui permet d'ajouter des livres, de gerer les emprunts, et de suivre les retours.

Cahier des charges

Fonctionnalite	Description
Ajouter un livre	Titre, auteur, annee
Lister les livres	Tous ou disponibles
Emprunter un livre	Marquer comme emprunte
Retourner un livre	Marquer comme disponible
Rechercher	Par titre ou auteur
Statistiques	Livres empruntes

Concepts Rust utilises

Concept	Utilisation
Vec	Stockage des livres
String	Titres et auteurs
Struct	Livre avec etat
Option	Emprunteur optionnel
Ownership	Transfert des noms
Borrowing	Recherche et affichage

Code complet

```
// =====
// STRUCTURES
// =====

#[derive(Debug)]
struct Livre {
    id: usize,
    titre: String,
    auteur: String,
    annee: u32,
    emprunteur: Option<String>,
}

impl Livre {
    fn nouveau(id: usize, titre: String, auteur: String, annee: u32) -> Livre {
        Livre {
            id,
            titre,
            auteur,
            annee,
            emprunteur: None,
        }
    }

    fn est_disponible(&self) -> bool {
        self.emprunteur.is_none()
    }

    fn emprunter(&mut self, nom: String) -> bool {
        if self.est_disponible() {
            self.emprunteur = Some(nom);
            true
        } else {
            false
        }
    }

    fn retourner(&mut self) -> bool {
        if self.emprunteur.is_some() {
```

```

        self.emprunteur = None;
        true
    } else {
        false
    }
}

fn afficher(&self) {
    let statut = if self.est_disponible() {
        "DISPONIBLE"
    } else {
        match &self.emprunteur {
            Some(nom) => &format!("Emprunte par {}", nom),
            None => "INDISPONIBLE",
        }
    };

    println!(" {:>4} | \"{}\" par {} ({{}} | {{}}",
             self.id, self.titre, self.auteur, self.annee, statut);
}

}

// =====
// FONCTIONS
// =====

fn ajouter_livre(bibliotheque: &mut Vec<Livre>, titre: String, auteur: String, annee: u32, next_id:
    => &mut usize) {
    bibliotheque.push(Livre::nouveau(*next_id, titre, auteur, annee));
    *next_id += 1;
}

fn lister_livres(bibliotheque: &Vec<Livre>, uniquement_disponibles: bool) {
    println!("\n=== Catalogue ===");
    for livre in bibliotheque {
        if uniquement_disponibles && !livre.est_disponible() {
            continue;
        }
        livre.afficher();
    }
    println!("=====\n");
}

fn trouver_livre_mut<'a>(bibliotheque: &'a mut Vec<Livre>, id: usize) -> Option<&'a mut Livre> {
    bibliotheque.iter_mut().find(|l| l.id == id)
}

fn trouver_livre<'a>(bibliotheque: &'a Vec<Livre>, id: usize) -> Option<&'a Livre> {
    bibliotheque.iter().find(|l| l.id == id)
}

```

```

fn rechercher_par_titre<'a>(bibliotheque: &'a Vec<Livres>, recherche: &str) -> Vec<&'a Livre> {
    bibliotheque.iter()
        .filter(|l| l.titre.to_lowercase().contains(&recherche.to_lowercase()))
        .collect()
}

fn rechercher_par_auteur<'a>(bibliotheque: &'a Vec<Livres>, recherche: &str) -> Vec<&'a Livre> {
    bibliotheque.iter()
        .filter(|l| l.auteur.to_lowercase().contains(&recherche.to_lowercase()))
        .collect()
}

fn statistiques(bibliotheque: &Vec<Livres>) {
    let total = bibliotheque.len();
    let empruntes = bibliotheque.iter().filter(|l| !l.est_disponible()).count();

    println!("\n=== Statistiques ===");
    println!(" Total : {} livres", total);
    println!(" Disponibles : {}", total - empruntes);
    println!(" Empruntes : {}", empruntes);
    println!("=====\n");
}

// =====
// MAIN
// =====

fn main() {
    let mut bibliotheque: Vec<Livres> = Vec::new();
    let mut next_id: usize = 1;

    println!("=== Gestionnaire de Bibliotheque ===\n");

    // Livres de demonstration
    ajouter_livre(&mut bibliotheque, String::from("Le Petit Prince"),
        String::from("Antoine de Saint-Exupery"), 1943, &mut next_id);
    ajouter_livre(&mut bibliotheque, String::from("1984"),
        String::from("George Orwell"), 1949, &mut next_id);
    ajouter_livre(&mut bibliotheque, String::from("Dune"),
        String::from("Frank Herbert"), 1965, &mut next_id);
    ajouter_livre(&mut bibliotheque, String::from("Fondation"),
        String::from("Isaac Asimov"), 1951, &mut next_id);
    ajouter_livre(&mut bibliotheque, String::from("Neuromancien"),
        String::from("William Gibson"), 1984, &mut next_id);

    // Affichage complet
    lister_livres(&bibliotheque, false);

    // Emprunt
    println!("--- Emprunt du livre 1 ---");
    if let Some(livre) = trouver_livre_mut(&mut bibliotheque, 1) {

```

```

        if livre.emprunter(String::from("Alice Martin")) {
            println!("Emprunt reussi !");
        }
    }

    // Emprunt d'un autre
    if let Some(livre) = trouver_livre_mut(&mut bibliotheque, 3) {
        livre.emprunter(String::from("Bob Dupont"));
    }

    // Affichage apres emprunts
    lister_livres(&bibliotheque, false);

    // Disponibles uniquement
    println!("--- Livres disponibles ---");
    lister_livres(&bibliotheque, true);

    // Recherche
    println!("--- Recherche par titre : 'ne' ---");
    let resultats = rechercher_par_titre(&bibliotheque, "ne");
    for l in resultats {
        l.afficher();
    }

    // Retour
    println!("--- Retour du livre 1 ---");
    if let Some(livre) = trouver_livre_mut(&mut bibliotheque, 1) {
        if livre.retourner() {
            println!("Retour effectue !");
        }
    }

    // Statistiques
    statistiques(&bibliotheque);

    // Affichage final
    lister_livres(&bibliotheque, false);
}

```

Explication detaillee

Lignes 9-14 : `emprunteur: Option<String>` stocke le nom de l'emprunteur. `None` signifie que le livre est disponible.

Lignes 24-34 : `emprunter` verifie d'abord si le livre est disponible avant de l'emprunter. Elle retourne un `bool` pour indiquer le succes.

Lignes 81-86 : `rechercher_par_titre` convertit en minuscules pour une recherche insensible a la casse. Elle retourne un `Vec<&Livre>`.

Lignes 113-117 : Le pattern `if let Some(livre) = ...` est utilise pour deballer l'Option retournee par `trouver_livre_mut`.

Fonctions et bibliotheques

Fonction	Role
<code>Option<String></code>	Emprunteur optionnel
<code>.is_none()</code> / <code>.is_some()</code>	Teste l' Option
<code>if let Some(x) = opt</code>	Pattern matching simple
<code>.to_lowercase()</code>	Conversion minuscules

Ameliorations possibles

1. Dates d'emprunt et retour
2. Limitation du nombre d'emprunts par personne
3. Systeme de reservation
4. Categories litteraires
5. ISBN et codes-barres
6. Import/export CSV
7. Alertes de retard
8. Historique des emprunts
9. Multi-bibliotheques
10. Recherche avancee (full-text)

Defi expert

Ajoutez un systeme de dates avec chrono, des alertes pour les retards, et un historique complet des transactions.

Projet 4 : Mini Systeme Bancaire

Presentation

Application bancaire simplifiee permettant de gerer des comptes, effectuer des depots, retraits, et virements.

Cahier des charges

Fonctionnalite	Description
Creer un compte	Nom titulaire + solde initial
Deposer	Ajouter des fonds
Retirer	Retirer des fonds (avec verification)
Virer	Transfert entre comptes
Consulter solde	Afficher le solde
Releve	Historique des operations
Lister comptes	Tous les comptes

Concepts Rust utilises

Concept	Utilisation
Vec	Stockage des comptes
Struct	Compte bancaire
Enum	Type d' operation
Result	Gestion des erreurs
Ownership	Transferts de fonds
Mutable References	Modification des soldes

Code complet

```
// =====
// ENUMERATIONS
// =====

#[derive(Debug, Clone)]
enum TypeOperation {
    Depot,
    Retrait,
    VirementEntrant,
    VirementSortant,
}

// =====
// STRUCTURES
// =====

#[derive(Debug)]
struct Operation {
    type_op: TypeOperation,
    montant: f64,
    description: String,
}

#[derive(Debug)]
struct Compte {
    id: usize,
    titulaire: String,
    solde: f64,
    operations: Vec<Operation>,
}

impl Compte {
    fn nouveau(id: usize, titulaire: String, solde_initial: f64) -> Compte {
        let mut compte = Compte {
            id,
            titulaire,
            solde: 0.0,
            operations: Vec::new(),
        };
    }
}
```



```

        if solde_initial > 0.0 {
            compte.deposer(solde_initial, String::from("Ouverture de compte"));
        }

        compte
    }

fn deposer(&mut self, montant: f64, description: String) -> bool {
    if montant <= 0.0 {
        return false;
    }

    self.solde += montant;
    self.operations.push(Operation {
        type_op: TypeOperation::Depot,
        montant,
        description,
    });
    true
}

fn retirer(&mut self, montant: f64, description: String) -> bool {
    if montant <= 0.0 || montant > self.solde {
        return false;
    }

    self.solde -= montant;
    self.operations.push(Operation {
        type_op: TypeOperation::Retrait,
        montant,
        description,
    });
    true
}

fn virer_vers(&mut self, montant: f64, destinataire: &mut Compte, description: String) -> bool {
    if montant <= 0.0 || montant > self.solde {
        return false;
    }

    // Retrait de l'emetteur
    self.solde -= montant;
    self.operations.push(Operation {
        type_op: TypeOperation::VirementSortant,
        montant,
        description: format!("Virement vers {} : {}", destinataire.titulaire, description),
    });

    // Depot chez le destinataire
    destinataire.solde += montant;

```

```

        destinataire.operations.push(Operation {
            type_op: TypeOperation::VirementEntrant,
            montant,
            description: format!("Virement depuis {} : {}", self.titulaire, description),
        });

        true
    }

    fn afficher_releve(&self) {
        println!("\n--- Releve du compte {} ({} ) ---", self.id, self.titulaire);
        println!("  Solde actuel : {:.2} euros", self.solde);
        println!("  Operations :");

        for op in &self.operations {
            let signe = match op.type_op {
                TypeOperation::Depot | TypeOperation::VirementEntrant => "+",
                TypeOperation::Retrait | TypeOperation::VirementSortant => "-",
            };
            println!("    {} {:.>10.2} | {:.?} | {}",
                signe, op.montant, op.type_op, op.description);
        }
        println!("-----\n");
    }
}

// =====
// FONCTIONS
// =====

fn creer_compte(banque: &mut Vec<Compte>, titulaire: String, solde_initial: f64, next_id: &mut usize) {
    banque.push(Compte::nouveau(*next_id, titulaire, solde_initial));
    *next_id += 1;
}

fn trouver_compte_mut<'a>(banque: &'a mut Vec<Compte>, id: usize) -> Option<&'a mut Compte> {
    banque.iter_mut().find(|c| c.id == id)
}

fn trouver_compte<'a>(banque: &'a Vec<Compte>, id: usize) -> Option<&'a Compte> {
    banque.iter().find(|c| c.id == id)
}

fn lister_comptes(banque: &Vec<Compte>) {
    println!("\n=== Liste des comptes ===");
    for compte in banque {
        println!("  {:.>4} | {:.<20} | {:.>10.2} euros",
            compte.id, compte.titulaire, compte.solde);
    }
    println!("=====\n");
}

```

```

fn statistiques_banque(banque: &Vec<Compte>) {
    let total_depots: f64 = banque.iter().map(|c| c.solde).sum();
    let nb_comptes = banque.len();

    println!("\n=== Statistiques ===");
    println!(" Nombre de comptes : {}", nb_comptes);
    println!(" Encours total : {:.2} euros", total_depots);

    if nb_comptes > 0 {
        println!(" Solde moyen : {:.2} euros", total_depots / nb_comptes as f64);
    }
    println!("=====\n");
}

// =====
// MAIN
// =====

fn main() {
    let mut banque: Vec<Compte> = Vec::new();
    let mut next_id: usize = 1;

    println!("=== Mini Systeme Bancaire ===\n");

    // Creation de comptes
    creer_compte(&mut banque, String::from("Alice Martin"), 1000.0, &mut next_id);
    creer_compte(&mut banque, String::from("Bob Dupont"), 500.0, &mut next_id);
    creer_compte(&mut banque, String::from("Charlie Durand"), 200.0, &mut next_id);

    // Affichage initial
    lister_comptes(&banque);

    // Depot
    println!("--- Depot de 500 euros sur le compte 1 ---");
    if let Some(compte) = trouver_compte_mut(&mut banque, 1) {
        compte.deposer(500.0, String::from("Salaire"));
    }

    // Retrait
    println!("--- Retrait de 200 euros sur le compte 2 ---");
    if let Some(compte) = trouver_compte_mut(&mut banque, 2) {
        if compte.retirer(200.0, String::from("Courses")) {
            println!("Retrait reussi !");
        } else {
            println!("Retrait refuse (solde insuffisant)");
        }
    }

    // Virement
    println!("--- Virement de 300 euros du compte 1 vers le compte 3 ---");

```

```
{
    let compte1_id = 1;
    let compte3_id = 3;

    // Trouver les indices pour pouvoir emprunter simultanement
    let idx1 = banque.iter().position(|c| c.id == compte1_id).unwrap();
    let idx3 = banque.iter().position(|c| c.id == compte3_id).unwrap();

    // Split_at_mut pour obtenir deux emprunts mutables disjoints
    if idx1 < idx3 {
        let (gauche, droite) = banque.split_at_mut(idx3);
        gauche[idx1].virer_vers(300.0, &mut droite[0], String::from("Remboursement"));
    } else {
        let (gauche, droite) = banque.split_at_mut(idx1);
        droite[0].virer_vers(300.0, &mut gauche[idx3], String::from("Remboursement"));
    }
}

// Affichage final
lister_comptes(&banque);

// Releves
for compte in &banque {
    compte.afficher_releve();
}

// Statistiques
statistiques_banque(&banque);
}
```

Explication detaillee

Lignes 55-79 : virer_vers est la fonction la plus complexe. Elle modifie **deux comptes simultanement** : l’emetteur et le destinataire. Comme Rust interdit deux emprunts mutables simultanés sur le meme vecteur, on utilise split_at_mut dans le main pour obtenir deux regions mutables disjointes.

Lignes 131-145 : Le virement utilise split_at_mut pour obtenir deux references mutables disjointes sur le vecteur banque. On trouve d’abord les indices, puis on scinde le vecteur.

Ligne 52-54 : if montant <= 0.0 || montant > self.solde verifie la validite du retrait. Si le montant est negatif, nul, ou superieur au solde, l’operation est refusee.

Fonctions et bibliotheques

Fonction	Role
split_at_mut()	Deux emprunts mutables disjoints
Result<T, E>	Gestion d’ erreurs typee
format!()	Creation de String formatee
match exhaustif	Pattern matching complet

Ameliorations possibles

1. Numeros de compte IBAN
2. Decouvert autorise
3. Interets
4. Historique persistant
5. Multi-devises
6. Cartes bancaires virtuelles
7. Beneficiaires enregistres
8. Virements automatiques
9. Alertes de solde
10. Export des relevés PDF

Defi expert

Implementez un systeme de decouvert autorise avec agios, des virements programmes, et un systeme d'alertes par email.

Projet 5 : Gestionnaire d'Etudiants

Presentation

Systeme complet de gestion d'etudiants avec notes, matieres, moyennes ponderees, et statistiques avancees.

Cahier des charges

Fonctionnalite	Description
Inscrire un etudiant	Nom, prenom, numero
Ajouter des matieres	Nom, coefficient
Attribuer des notes	Par etudiant et matiere
Calculer moyennes	Ponderes par coefficients
Bulletin	Releve complet
Classement	Par moyenne generale
Statistiques	Moyennes, ecart-types

Code complet

```
// =====  
// STRUCTURES  
// =====  
  
#[derive(Debug, Clone)]  
struct Matiere {  
    nom: String,  
    coefficient: u32,  
}
```

```

#[derive(Debug)]
struct Note {
    matiere: String,
    valeur: f64,
}

#[derive(Debug)]
struct Etudiant {
    numero: String,
    nom: String,
    prenom: String,
    notes: Vec<Note>,
}

impl Etudiant {
    fn nouveau(numero: String, nom: String, prenom: String) -> Etudiant {
        Etudiant {
            numero,
            nom,
            prenom,
            notes: Vec::new(),
        }
    }

    fn ajouter_note(&mut self, matiere: String, valeur: f64) {
        self.notes.push(Note { matiere, valeur });
    }

    fn moyenne_ponderee(&self, matieres: &Vec<Matiere>) -> Option<f64> {
        if self.notes.is_empty() {
            return None;
        }

        let mut total_points: f64 = 0.0;
        let mut total_coefficients: u32 = 0;

        for note in &self.notes {
            if let Some(matiere) = matieres.iter().find(|m| m.nom == note.matiere) {
                total_points += note.valeur * matiere.coefficient as f64;
                total_coefficients += matiere.coefficient;
            }
        }

        if total_coefficients == 0 {
            None
        } else {
            Some(total_points / total_coefficients as f64)
        }
    }

    fn afficher_bulletin(&self, matieres: &Vec<Matiere>) {

```

```

println!("\n--- Bulletin ---");
println!("{}", self.prenom, self.nom, self.numero);
println!("{}", "-----");

for note in &self.notes {
    let coeff = matieres.iter()
        .find(|m| m.nom == note.matiere)
        .map(|m| m.coefficient)
        .unwrap_or(1);
    println!("{}", "{:<20} | {:>5.2} (coef. {})", note.matiere, note.valeur, coeff);
}

if let Some(moy) = self.moyenne_ponderee(matieres) {
    println!("{}", "-----");
    println!("{}", "MOYENNE GENERALE : {:.2}/20", moy);
}
println!("{}", "-----\n");
}
}

// =====
// FONCTIONS
// =====

fn inscrire_etudiant(etudiants: &mut Vec<Etudiant>, numero: String, nom: String, prenom: String) {
    etudiants.push(Etudiant::nouveau(numero, nom, prenom));
}

fn trouver_etudiant_mut<'a>(etudiants: &'a mut Vec<Etudiant>, numero: &str) -> Option<&'a mut Etudiant> {
    ↪ {
        etudiants.iter_mut().find(|e| e.numero == numero)
    }
}

fn trouver_etudiant<'a>(etudiants: &'a Vec<Etudiant>, numero: &str) -> Option<&'a Etudiant> {
    etudiants.iter().find(|e| e.numero == numero)
}

fn classement<'a>(etudiants: &'a Vec<Etudiant>, matieres: &Vec<Matiere>) -> Vec<(&'a str, f64)> {
    let mut resultat: Vec<(&str, f64)> = etudiants.iter()
        .filter_map(|e| {
            e.moyenne_ponderee(matieres).map(|moy| (e.numero.as_str(), moy))
        })
        .collect();

    resultat.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap());
    resultat
}

fn statistiques_globales(etudiants: &Vec<Etudiant>, matieres: &Vec<Matiere>) {
    let moyennes: Vec<f64> = etudiants.iter()
        .filter_map(|e| e.moyenne_ponderee(matieres))

```

```

        .collect();

    if moyennes.is_empty() {
        println!("Aucune moyenne disponible.");
        return;
    }

    let somme: f64 = moyennes.iter().sum();
    let moyenne_classe = somme / moyennes.len() as f64;

    // Ecart-type
    let variance: f64 = moyennes.iter()
        .map(|m| (m - moyenne_classe).powi(2))
        .sum::<f64>() / moyennes.len() as f64;
    let ecart_type = variance.sqrt();

    println!("\n=== Statistiques Globales ===");
    println!(" Nombre d'etudiants : {}", etudiants.len());
    println!(" Moyenne de la classe : {:.2}/20", moyenne_classe);
    println!(" Ecart-type : {:.2}", ecart_type);
    println!(" Meilleure moyenne : {:.2}", moyennes.iter().fold(0.0, |a, &b| a.max(b)));
    println!(" Moins bonne moyenne : {:.2}", moyennes.iter().fold(20.0, |a, &b| a.min(b)));
    println!(" Mediane : {:.2}", calculer_mediane(&moyennes));
    println!(" Taux de reussite (>10) : {:.1}%",
        moyennes.iter().filter(|&m| m >= 10.0).count() as f64 / moyennes.len() as f64 * 100.0);
    println!("=====\n");
}

fn calculer_mediane(valeurs: &Vec<f64>) -> f64 {
    let mut trie = valeurs.clone();
    trie.sort_by(|a, b| a.partial_cmp(b).unwrap());

    let milieu = trie.len() / 2;
    if trie.len() % 2 == 0 {
        (trie[milieu - 1] + trie[milieu]) / 2.0
    } else {
        trie[milieu]
    }
}

fn afficher_classement(etudiants: &Vec<Etudiant>, matieres: &Vec<Matiere>) {
    println!("\n=== Classement ===");
    let classe = classement(etudiants, matieres);

    for (i, (numero, moy)) in classe.iter().enumerate() {
        if let Some(etudiant) = trouver_etudiant(etudiants, numero) {
            println!(" {} {} {} - {:.2}/20",
                i + 1, etudiant.prenom, etudiant.nom, moy);
        }
    }
    println!("=====\n");
}

```



```

}

// =====
// MAIN
// =====

fn main() {
    let mut etudiants: Vec<Etudiant> = Vec::new();
    let mut matieres: Vec<Matiere> = Vec::new();

    println!("=== Gestionnaire d'Etudiants ===\n");

    // Definition des matieres avec coefficients
    matieres.push(Matiere { nom: String::from("Mathematiques"), coefficient: 4 });
    matieres.push(Matiere { nom: String::from("Physique"), coefficient: 3 });
    matieres.push(Matiere { nom: String::from("Informatique"), coefficient: 3 });
    matieres.push(Matiere { nom: String::from("Anglais"), coefficient: 2 });
    matieres.push(Matiere { nom: String::from("Francais"), coefficient: 2 });

    // Inscription des etudiants
    inscrire_etudiant(&mut etudiants, String::from("E001"),
        String::from("Martin"), String::from("Alice"));
    inscrire_etudiant(&mut etudiants, String::from("E002"),
        String::from("Dupont"), String::from("Bob"));
    inscrire_etudiant(&mut etudiants, String::from("E003"),
        String::from("Durand"), String::from("Charlie"));

    // Attribution des notes a Alice
    if let Some(e) = trouver_etudiant_mut(&mut etudiants, "E001") {
        e.ajouter_note(String::from("Mathematiques"), 16.0);
        e.ajouter_note(String::from("Physique"), 14.5);
        e.ajouter_note(String::from("Informatique"), 18.0);
        e.ajouter_note(String::from("Anglais"), 15.0);
        e.ajouter_note(String::from("Francais"), 13.5);
    }

    // Attribution des notes a Bob
    if let Some(e) = trouver_etudiant_mut(&mut etudiants, "E002") {
        e.ajouter_note(String::from("Mathematiques"), 11.0);
        e.ajouter_note(String::from("Physique"), 9.5);
        e.ajouter_note(String::from("Informatique"), 12.0);
        e.ajouter_note(String::from("Anglais"), 14.0);
        e.ajouter_note(String::from("Francais"), 10.5);
    }

    // Attribution des notes a Charlie
    if let Some(e) = trouver_etudiant_mut(&mut etudiants, "E003") {
        e.ajouter_note(String::from("Mathematiques"), 19.0);
        e.ajouter_note(String::from("Physique"), 17.5);
        e.ajouter_note(String::from("Informatique"), 18.5);
        e.ajouter_note(String::from("Anglais"), 16.0);
    }
}

```

```
        e.ajouter_note(String::from("Francais"), 17.0);
    }

    // Bulletins
    for etudiant in &etudiants {
        etudiant.afficher_bulletin(&matieres);
    }

    // Classement
    afficher_classement(&etudiants, &matieres);

    // Statistiques
    statistiques_globales(&etudiants, &matieres);
}
```

Explication detaillee

Lignes 30-48 : `moyenne_ponderee` calcule la moyenne en tenant compte des coefficients. Elle retourne `Option<f64>` car l’etudiant pourrait ne pas avoir de notes.

Lignes 92-102 : `classement` cree un vecteur de tuples (`numero`, `moyenne`), le trie par moyenne decroissante, et le retourne. Le `filter_map` combine le filtrage et la transformation.

Lignes 106-126 : `statistiques_globales` calcule la moyenne, l’ecart-type, la mediane, et le taux de reussite. Elle utilise `.powi(2)` pour le carre et `.sqrt()` pour la racine carree.

Lignes 128-139 : `calculer_mediane` clone le vecteur, le trie, et retourne la valeur centrale (ou la moyenne des deux valeurs centrales).

Fonctions et bibliotheques

Fonction	Role
<code>filter_map()</code>	Filtre et transforme en une operation
<code>fold()</code>	Reduction d’ un iterateur
<code>partial_cmp()</code>	Comparaison pour les flottants
<code>powi()</code>	Puissance entiere
<code>sqrt()</code>	Racine carree
<code>min()</code> / <code>max()</code>	Minimum et maximum

Ameliorations possibles

- 1. Annees scolaires
- 2. Semestres
- 3. UE et matieres optionnelles
- 4. Rattrapages
- 5. Jurys
- 6. Export de releves
- 7. Absences et retard
- 8. Commentaires des professeurs
- 9. Evolution dans le temps

10. Predictions de mentions

Defi expert

Implementez un systeme complet avec annees scolaires, semestres, matieres optionnelles, et generation automatique de PV de jury avec decision.

Conclusion

Recapitulatif des Concepts Acquis

Concept	Exercices	Projets
Variables et types	1, 2	Tous
Fonctions	1, 2	Tous
Controle de flux	1, 2, 4, 5	Tous
Vec	6, 8, 9	Tous
String	7, 10	Tous
Ownership	11, 19	Tous
Borrowing immuable	12	Tous
Borrowing mutable	14	Tous
Slices	13, 16, 17, 18	Tous
References	12, 13, 14	Tous
Structs	Projet 1-5	Tous
Enums	Projet 2, 4	Projets 2, 4
Iterateurs	6, 7, 8, 9	Tous
Pattern matching	4, 9, 15	Tous

Prochaines Etapes

Apres avoir maitrise ces exercices et projets, vous etes pret a aborder :

- Les **traits** et les **generiques**
- La **gestion des erreurs** avancee avec **Result**
- Les **fermetures** (closures)
- Les **iterateurs** personnalisés
- La **programmation asynchrone**
- Les **macros**
- La **concurrency** avec les threads

Ressources Recommandees

Ressource	Description
The Rust Programming Language	Le livre officiel (gratuit en ligne)
Rust by Example	Exemples pratiques par concept
Rustlings	Petits exercices interactifs

Table 60 – continued

Ressource	Description
Exercism Rust Track	Exercices progressifs avec mentorat

Fin du Cahier d'Exercices

Ce document a été conçu pour vous accompagner dans votre apprentissage pratique de Rust.
Pratiquez régulièrement, lisez les erreurs du compilateur, et n'hésitez pas à expérimenter.